

Módulo 1 - Guía práctica

Curso Spring Boot 2026

Índice

Módulo 1 - Guía práctica: de una aplicación que funciona a una API que entendemos	6
Qué vas a construir y comprobar	6
Práctica 1.1 - Observar Spring Boot: auto-configuración, starters y servidor embebido	6
Objetivo	6
Requisitos previos	6
Paso 1 - Abrir el proyecto y arrancarlo	7
Qué hacemos	7
Desde IntelliJ IDEA	7
Desde Eclipse	7
Desde VS Code	8
Qué debes razonar	8
Pregunta	8
Respuesta razonada	8
Paso 2 - Ver el informe de auto-configuración	8
Qué hacemos	8
IntelliJ IDEA	8
Eclipse	9
VS Code	9
Qué debes localizar	9
Pregunta	9
Respuesta razonada	9
Paso 3 - Leer el informe con calma	9
Qué hacemos	9
Construye una tabla de observación	10
Qué debes entender	10
Pregunta	10
Respuesta razonada	10
Paso 4 - Demostrar que Tomcat está embebido	10
4.1 Comprobar si existe una instalación externa	10
4.2 Demostrar que Tomcat pertenece al proyecto	11
4.3 Demostrar el modelo ejecutable	11
Pregunta	11
Respuesta razonada	11
Paso 5 - Cambiar una configuración por defecto y restaurarla	11
Objetivo del experimento	11
Qué has demostrado	12
Restauración obligatoria	12
Pregunta	12
Respuesta razonada	12
Paso 6 - Observar los starters y las dependencias transitivas	13
Ver el árbol real	13
En IntelliJ IDEA	13
En Eclipse	13
En VS Code	13

Pregunta	13
Respuesta razonada	13
Paso 7 - Probar /hola y reconstruir toda la cadena	14
Relación con Spring Boot	14
Pregunta	14
Respuesta razonada	14
Paso 8 - Explicar el papel de Spring Boot usando evidencias del proyecto	14
Pregunta	15
Respuesta razonada	15
Paso 9 - Diagnosticar errores frecuentes sin cambiar varias cosas a la vez	15
Método de diagnóstico	15
Pregunta	16
Respuesta razonada	16
Paso 10 - Reto resuelto: excluir temporalmente Jackson y demostrar el efecto	16
El problema del reto original	16
Estado inicial que debes conservar	16
10.1 Añadir un probe JSON temporal	16
10.2 Excluir temporalmente la auto-configuración de Jackson	17
10.3 Comprobar que /hola no es una prueba suficiente de Jackson	17
10.4 Restaurar MiProyectoApplication	17
10.5 Eliminar el probe temporal	17
10.6 Verificación de restauración	18
Qué has demostrado	18
Pregunta	18
Respuesta razonada	18
Resultado esperado global de la práctica 1.1	18
Checklist de cierre	19

Práctica 1.2 - Observar HTTP con navegador, DevTools y curl **19**

Objetivo	19
Paso 1 - Arrancar la aplicación y formular hipótesis	19
Qué hacemos	19
Por qué empezamos así	20
Pregunta	20
Respuesta razonada	20
Paso 2 - Probar GET con el navegador	20
Reconstruye lo que ha ocurrido	20
Pregunta	20
Respuesta razonada	21
Paso 3 - Probar una ruta que no existe	21
Qué significa	21
Pregunta	21
Respuesta razonada	21
Paso 4 - Probar GET con curl y ver las cabeceras	21
Pregunta	22
Respuesta razonada	22
Paso 5 - Probar una ruta inexistente con curl	22
Pregunta	22
Respuesta razonada	22
Paso 6 - Probar un método HTTP no soportado	22
Pregunta	23
Respuesta razonada	23
Paso 7 - Ver las cabeceras que envía el cliente con curl	23
Pregunta	23
Respuesta razonada	23
Paso 8 - Observar la misma petición con DevTools	23
Cuatro entornos	24
Pregunta	24

Respuesta razonada	24
Paso 9 - Razonar sobre lo observado	24
Pregunta	24
Respuesta razonada	25
Paso 10 - Diagnosticar errores comunes de comunicación HTTP	25
Regla de diagnóstico	25
Pregunta	25
Respuesta razonada	25
Paso 11 - Resumir lo observado con pruebas reproducibles	25
Pregunta	26
Respuesta razonada	26
Paso 12 - Reto resuelto: provocar un 400 Bad Request y restaurar el proyecto	26
Objetivo	26
12.1 Baseline	26
12.2 Añadir temporalmente /eco	26
12.3 Probar JSON mal formado	27
12.4 Probar JSON válido	27
12.5 Diferenciar 400 de 415	27
12.6 Restauración obligatoria	27
Pregunta	28
Respuesta razonada	28
Resultado esperado global de la práctica 1.2	28
Checklist de cierre	28

Práctica 1.3 - Jackson y DTOs 28

Objetivo	28
Paso 1 - Crear el paquete dto	29
Qué hacemos	29
Consola	29
IntelliJ IDEA	29
Eclipse	29
VS Code	29
Por qué un paquete específico	29
Pregunta	30
Respuesta razonada	30
Paso 2 - Crear la clase ExpedienteDTO	30
Qué debes observar	32
Pregunta	32
Respuesta razonada	32
Paso 3 - Crear un controlador para probar la serialización	33
Antes de llamar al endpoint	33
Pregunta	33
Respuesta razonada	33
Paso 4 - Arrancar y probar la serialización por defecto	34
Corrección respecto de material antiguo	34
Pregunta	34
Respuesta razonada	34
Paso 5 - Renombrar el campo con @JsonProperty	34
Qué no ha cambiado	35
Pregunta	35
Respuesta razonada	35
Paso 6 - Formatear la fecha con @JsonFormat	35
Qué demuestra realmente este paso	35
Pregunta	35
Respuesta razonada	36
Paso 7 - Omitir campos nulos con @JsonInclude	36
Pregunta	36
Respuesta razonada	36

Paso 8 - Excluir un campo con @JsonIgnore	36
Pregunta	37
Respuesta razonada	37
Paso 9 - Probar la deserialización con POST	37
Sigue el recorrido	38
Prueba también una fecha inválida	38
Pregunta	38
Respuesta razonada	38
Paso 10 - Probar la deserialización con campos desconocidos	38
Precisión importante	38
Prueba @JsonIgnore también en entrada	38
Pregunta	39
Respuesta razonada	39
Paso 11 - Errores comunes del ejercicio	39
Estrategia	39
Pregunta	39
Respuesta razonada	40
Paso 12 - Reto resuelto: DTO con DTO anidado	40
12.1 Crear SolicitanteDTO	40
12.2 Añadirlo a ExpedienteDTO	41
12.3 Evolucionar /ejemplo	41
12.4 Probar serialización anidada	41
12.5 Probar deserialización anidada	42
Cierre técnico	42
Pregunta	42
Respuesta razonada	42
Resultado esperado global de la práctica 1.3	42
Checklist de cierre	43
Práctica 1.4 - Diseño de APIs REST	43
Objetivo práctico	43
Estado de partida	43
Desde IntelliJ IDEA	43
Desde Eclipse	43
Desde VS Code	44
Paso 1 - Identificar los recursos y sus operaciones	44
Qué debes observar	44
Pregunta	44
Respuesta razonada	44
Paso 2 - Crear el DTO AlumnoDTO	44
Por qué reutilizamos las anotaciones de 1.3	46
Pregunta	46
Respuesta razonada	46
Paso 3 - Crear AlumnoController con GET colección y POST básico	47
Qué significa este estado	47
Pregunta	47
Respuesta razonada	48
Paso 4 - Arrancar y probar el GET de colección	48
Pregunta	48
Respuesta razonada	48
Paso 5 - Probar el POST de creación y observar el 200 inicial	48
Pregunta	48
Respuesta razonada	48
Paso 6 - Analizar por qué 200 no describe una creación	49
Pregunta	49
Respuesta razonada	49
Paso 7 - Corregir el POST para devolver 201 Created	49
Pregunta	49

Respuesta razonada	49
Paso 8 - Añadir el endpoint GET individual	50
Qué hace @PathVariable	50
Pregunta	50
Respuesta razonada	50
Paso 9 - Probar un ID inexistente	50
Pregunta	51
Respuesta razonada	51
Paso 10 - Auditar el diseño actual de la API	51
Pregunta	51
Respuesta razonada	51
Paso 11 - Reconocer errores comunes de diseño	51
Diagnóstico operativo	52
Pregunta	52
Respuesta razonada	52
Paso 12 - Reto resuelto: filtrar por curso con query parameter	52
Pregunta	52
Respuesta razonada	53
Resultado esperado al cerrar 1.4	53

Práctica 1.5 - Primer CRUD con DTOs **53**

Objetivo práctico	53
Estado de partida	53
Desde IntelliJ IDEA	54
Desde Eclipse	54
Desde VS Code	54
Paso 1 - Repasar el estado actual	54
Pregunta	54
Respuesta razonada	54
Paso 2 - Convertir la lista en mutable	54
Pregunta	55
Respuesta razonada	55
Paso 3 - Modificar el POST para que guarde el alumno	55
Pregunta	56
Respuesta razonada	56
Paso 4 - Añadir el endpoint PUT	56
Pregunta	56
Respuesta razonada	57
Paso 5 - Añadir el endpoint PATCH	57
Pregunta	58
Respuesta razonada	58
Paso 6 - Añadir el endpoint DELETE	58
Pregunta	58
Respuesta razonada	58
Paso 7 - Probar el CRUD completo con curl	58
1. Listar	58
2. Consultar	59
3. Crear	59
4. Consultar el creado	59
5. Reemplazar con PUT	59
6. Modificar parcialmente con PATCH	59
7. Eliminar	59
8. Verificar ausencia	59
Pregunta	59
Respuesta razonada	59
Paso 8 - Observar los códigos de estado	59
Pregunta	60
Respuesta razonada	60

Paso 9 - Probar un error de formato	60
Pregunta	60
Respuesta razonada	60
Paso 10 - Probar un método no soportado	61
Pregunta	61
Respuesta razonada	61
Paso 11 - Errores comunes del ejercicio	61
Límite importante	61
Pregunta	61
Respuesta razonada	62
Paso 12 - Reto resuelto: filtrar por curso y ordenar	62
Pregunta	63
Respuesta razonada	63
Resultado esperado al cerrar 1.5 y M1	63

Módulo 1 - Guía práctica: de una aplicación que funciona a una API que entendemos

Qué vas a construir y comprobar

M1 continúa exactamente desde el proyecto final aprobado de M0. No comenzamos con un proyecto diferente ni repetimos Spring Initializr. Vamos a utilizar el código que ya funciona para descubrir qué infraestructura hay detrás y, en los siguientes puntos, transformarlo progresivamente en una API REST con JSON y un primer CRUD.

En el punto 1.1 no añadiremos funcionalidad permanente. Haremos varios experimentos controlados y al terminar el proyecto volverá al estado funcional heredado de M0. Esta disciplina es deliberada: un experimento temporal debe poder demostrarse y después cerrarse sin dejar configuración accidental.

El snapshot acumulativo del módulo está en M1/proyecto/.

Baseline del curso: Java 17, Maven 3.9.x mediante Maven Wrapper 3.9.16 y Spring Boot 3.5.16.

Práctica 1.1 - Observar Spring Boot: auto-configuración, starters y servidor embebido

Objetivo

Al terminar esta práctica habrás demostrado sobre tu proyecto real que:

- Spring Boot ha preparado infraestructura que tú no configuraste manualmente;
- esa infraestructura depende del classpath, de condiciones y de propiedades;
- los starters explican por qué aparecen Spring MVC, Jackson y Tomcat;
- Tomcat forma parte del runtime de la aplicación y no necesita una instalación externa;
- los valores por defecto se pueden sobrescribir;
- el informe de condiciones permite diagnosticar decisiones de auto-configuración;
- una exclusión temporal de auto-configuración cambia comportamiento observable y puede restaurarse limpiamente.

Requisitos previos

Debes partir del proyecto final de M0. Antes de comenzar, comprueba que existen al menos:

M1/proyecto/
|-- pom.xml

```
| -- mvnw
| -- mvnw.cmd
| -- src/
|   |-- main/
|   |   |-- java/es/mecd/demo/miproyecto/
|   |   |   |-- MiProyectoApplication.java
|   |   |   |-- controller/SaludoController.java
|   |   |-- resources/application.properties
|   |-- test/
|   |   |-- java/es/mecd/demo/miproyecto/
|   |   |   |-- MiProyectoApplicationTests.java
|   |   |   |-- controller/SaludoControllerTest.java
```

SaludoController debe seguir exponiendo GET /hola y GET /adios.

Paso 1 - Abrir el proyecto y arrancarlo

Qué hacemos

Vamos a obtener una línea base antes de modificar nada. Necesitamos saber que el proyecto heredado funciona para poder atribuir después cualquier cambio al experimento que hagamos.

Desde la carpeta M1/proyecto, en Linux/macOS:

```
./mvnw spring-boot:run
```

En Windows PowerShell o CMD:

```
mvnw.cmd spring-boot:run
```

Espera a que aparezca en los logs una evidencia equivalente a:

```
Tomcat started on port 8080
Started MiProyectoApplication
```

La redacción exacta y los tiempos pueden variar. Lo importante es comprobar que el contexto termina de arrancar y que Tomcat publica el puerto.

En otra terminal:

```
curl -i http://localhost:8080/hola
curl -i http://localhost:8080/adios
```

Debes obtener 200 en ambos casos y los cuerpos heredados de M0.

Desde IntelliJ IDEA

1. Abre M1/proyecto como proyecto Maven.
2. Comprueba que el Project SDK es Java 17.
3. Abre MiProyectoApplication.java.
4. Ejecuta el botón Run junto al método main o la clase.
5. Observa la ventana Run.

Desde Eclipse

1. Importa M1/proyecto mediante **File > Import > Maven > Existing Maven Projects** si aún no está importado.
2. Abre MiProyectoApplication.java.
3. **Run As > Java Application.**
4. Observa la vista Console.

Desde VS Code

1. Abre la carpeta M1/proyecto.
2. Espera a que Java/Maven termine de cargar el proyecto.
3. Abre `MiProyectoApplication.java`.
4. Utiliza **Run Java** sobre `main` o la vista Spring Boot Dashboard si está disponible.
5. Observa la terminal o consola asociada.

Qué debes razonar

Antes de seguir, anota qué piezas crees que han intervenido para que `/hola` funcione. Como mínimo intenta responder:

- ¿quién escucha en el puerto 8080?;
- ¿quién decide que `/hola` corresponde a un método Java?;
- ¿de dónde salen las librerías necesarias?;
- ¿qué parte has configurado tú explícitamente y qué parte no?;

No busques todavía la respuesta “correcta”. En los pasos siguientes contrastaremos tus hipótesis.

Pregunta

¿Por qué comprobamos `/hola` y `/adios` antes de activar `--debug` o cambiar propiedades?

Respuesta razonada

Porque necesitamos una referencia. Si después aparece un fallo, podremos distinguir si ya existía antes del experimento o si lo introdujimos nosotros. Una prueba de baseline reduce diagnósticos ambiguos.

Paso 2 - Ver el informe de auto-configuración

Qué hacemos

Vamos a arrancar la misma aplicación con `--debug`. Ese argumento hace que Spring Boot muestre información adicional de diagnóstico, incluido el **Conditions Evaluation Report**.

Detén primero la aplicación anterior con `Ctrl+C` o con `Stop/Terminate` en el IDE.

Linux/macOS:

```
./mvnw spring-boot:run -Dspring-boot.run.arguments=--debug
```

Windows:

```
mvnw.cmd spring-boot:run -Dspring-boot.run.arguments=--debug
```

Si tu shell interpreta de forma especial algún carácter, utiliza la terminal integrada de tu IDE o configura `--debug` como argumento de programa.

IntelliJ IDEA

1. **Run > Edit Configurations.**
2. Selecciona la configuración de `MiProyectoApplication`.
3. En **Program arguments**, escribe:

```
--debug
```

4. Ejecuta de nuevo.

Eclipse

1. **Run > Run Configurations.**
2. Selecciona la configuración Java de MiProyectoApplication.
3. Pestaña **Arguments.**
4. En **Program arguments**, escribe `--debug`.
5. Ejecuta.

VS Code

Si usas una configuración `launch.json`, añade temporalmente:

"args": "--debug"

También puedes evitar tocar configuración del editor y ejecutar con Maven desde la terminal integrada.

Qué debes localizar

Busca en la consola:

CONDITIONS EVALUATION REPORT

Después localiza secciones equivalentes a:

Positive matches

Negative matches

Exclusions

No necesitas leer todavía cientos de líneas. El objetivo de este paso es demostrar que Spring Boot puede explicar muchas de sus decisiones.

Pregunta

¿Un Negative match significa que la aplicación tiene un error?

Respuesta razonada

No. Significa que una configuración candidata no cumplió alguna condición. Si no utilizamos una tecnología concreta, es perfectamente normal que su auto-configuración no se active. El informe describe decisiones, no sólo fallos.

Paso 3 - Leer el informe con calma

Qué hacemos

Ahora sí vamos a buscar evidencias concretas relacionadas con nuestro proyecto web.

En **Positive matches**, busca términos como:

DispatcherServletAutoConfiguration

ServletWebServerFactoryAutoConfiguration

Tomcat

La redacción exacta de cada condición puede variar entre versiones. No copies mecánicamente una salida de Internet: utiliza el informe producido por **tu** aplicación.

Lo que quieres demostrar es la relación entre clases disponibles y configuración activada.

Por ejemplo, conceptualmente puedes encontrar condiciones equivalentes a:

... matched because a required web class is present
 ... matched because this is a servlet web application
 ... matched because Tomcat classes are available

Después ve a **Negative matches** y busca alguna tecnología que no formes parte de este proyecto. Es normal encontrar numerosas configuraciones que no se aplican porque falta una clase, una propiedad o un bean.

Construye una tabla de observación

Rellena una tabla como esta con ejemplos reales de tu consola:

Configuración	Positive/Negative	Condición que observaste	Qué significa
DispatcherServlet...			
ServletWebServer...			
Tomcat...			
Otra no usada			

Qué debes entender

El informe nos permite pasar de:

“Spring Boot hace magia”

A:

“Spring Boot evaluó condiciones concretas sobre este classpath y este contexto, y por eso activó o descartó configuraciones”.

Pregunta

¿Qué cambiaría en este informe si elimináramos del POM una dependencia que introduce clases web esenciales?

Respuesta razonada

Cambiaría el classpath y, por tanto, podrían dejar de cumplirse condiciones que dependen de esas clases. Algunas configuraciones web pasarían a no aplicarse. Esta es una de las conexiones más importantes entre Maven y el runtime de Spring Boot.

Paso 4 - Demostrar que Tomcat está embebido

La fuente original propone comprobar si existe un comando tomcat instalado. Esa observación es útil, pero por sí sola no demuestra qué servidor utiliza nuestra aplicación. Vamos a hacer una verificación más fuerte.

4.1 Comprobar si existe una instalación externa

Linux/macOS:

```
which tomcat
```

Windows PowerShell:

```
Get-Command tomcat -ErrorAction SilentlyContinue
```

Si no aparece nada, es una situación habitual. Si aparece una ruta, tampoco invalida el ejercicio: sólo significa que tu máquina tiene alguna instalación o comando externo relacionado.

4.2 Demostrar que Tomcat pertenece al proyecto

Ejecuta:

Linux/macOS:

```
./mvnw dependency:tree
```

Windows:

```
mvnw.cmd dependency:tree
```

Busca tomcat en la salida. Debes encontrar dependencias de Tomcat introducidas por el stack web del proyecto.

Puedes filtrar visualmente o utilizar las herramientas de tu sistema. Por ejemplo:

Linux/macOS:

```
./mvnw dependency:tree | grep -i tomcat
```

PowerShell:

```
mvnw.cmd dependency:tree | Select-String -Pattern "tomcat"
```

4.3 Demostrar el modelo ejecutable

Empaqueta:

```
./mvnw clean package
```

En Windows:

```
mvnw.cmd clean package
```

Después arranca el JAR generado:

```
java -jar target/mi-proyecto-0.0.1-SNAPSHOT.jar
```

Y comprueba:

```
curl -i http://localhost:8080/hola
```

Si el JAR arranca el servidor y atiende la petición sin desplegarse manualmente en un Tomcat externo, has demostrado el modelo de servidor embebido de forma mucho más sólida que comprobando únicamente la ausencia de un programa instalado.

Pregunta

Si tu ordenador sí tiene un Tomcat instalado externamente, ¿cómo sabemos que este ejercicio no lo está utilizando?

Respuesta razonada

Porque el árbol de dependencias del proyecto contiene el runtime de Tomcat y el JAR ejecutable arranca directamente con `java -jar`. No hemos copiado un WAR a un `webapps/` externo ni hemos iniciado una instalación global. El ciclo de vida del servidor pertenece al proceso de la aplicación.

Paso 5 - Cambiar una configuración por defecto y restaurarla

Objetivo del experimento

Spring Boot utiliza por defecto el puerto 8080 en este proyecto. Vamos a sobrescribirlo con configuración externa y después volver al baseline.

Detén la aplicación.

Abre:

```
src/main/resources/application.properties
```

Su estado base contiene:

```
spring.application.name=mi-proyecto
```

Añade temporalmente:

```
server.port=9090
```

El archivo queda momentáneamente:

```
spring.application.name=mi-proyecto
server.port=9090
```

Arranca la aplicación.

Comprueba:

```
curl -i http://localhost:9090/hola
```

Debe responder 200.

Ahora prueba:

```
curl -i http://localhost:8080/hola
```

Si no existe otro proceso escuchando en 8080, esa conexión ya no debe llegar a nuestra aplicación.

Qué has demostrado

El servidor no está rígidamente fijado a 8080 en código Java. Spring Boot proporciona un valor convencional y `server.port` lo sobrescribe.

Restauración obligatoria

Detén la aplicación y elimina la línea temporal:

```
server.port=9090
```

El estado final vuelve a ser:

```
spring.application.name=mi-proyecto
```

Arranca de nuevo y verifica:

```
curl -i http://localhost:8080/hola
```

Debe volver a responder 200.

Pregunta

¿Por qué no dejamos 9090 como nuevo puerto “ya que también funciona”?

Respuesta razonada

Porque el objetivo era demostrar una sobrescritura, no cambiar el contrato del curso. Los siguientes ejercicios parten de 8080. Mantener cambios de laboratorio sin necesidad hace que las instrucciones futuras diverjan entre alumnos.

Paso 6 - Observar los starters y las dependencias transitivas

Abre pom.xml y localiza como mínimo:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

El proyecto también conserva spring-boot-devtools como ayuda de desarrollo heredada de M0.

Ver el árbol real

Ejecuta:

```
./mvnw dependency:tree
```

Localiza en el árbol elementos relacionados con:

- Spring MVC / Spring Web;
- Jackson;
- Tomcat;
- logging;
- JUnit y otras herramientas de test bajo el starter de test.

No necesitas memorizar el árbol entero. Queremos responder a una pregunta concreta: **¿por qué podemos importar y usar determinadas clases aunque no exista una dependencia directa para cada una en nuestro POM?**

En IntelliJ IDEA

Puedes utilizar la ventana Maven y su vista de dependencias. Según la versión del IDE, también puedes navegar desde el POM a la dependencia o utilizar un diagrama de dependencias Maven.

En Eclipse

Abre pom.xml y utiliza las vistas **Dependency Hierarchy** / **Dependencies** del editor Maven cuando estén disponibles.

En VS Code

La vista Maven puede mostrar dependencias; para una evidencia reproducible entre todos los entornos, dependency:tree sigue siendo la referencia común.

Pregunta

¿Por qué preferimos dependency:tree como evidencia común del curso aunque un IDE tenga una vista gráfica más cómoda?

Respuesta razonada

Porque el comando pertenece al sistema de construcción y produce una evidencia independiente del IDE. Las vistas gráficas son útiles, pero pueden variar entre herramientas y versiones. El árbol de Maven describe la resolución que utiliza el proyecto.

Paso 7 - Probar /hola y reconstruir toda la cadena

Con la aplicación otra vez en 8080, ejecuta:

```
curl -i http://localhost:8080/hola
```

No te limites a observar el texto. Reconstruye el recorrido:

1. curl actúa como cliente y abre una conexión al puerto 8080.
2. Tomcat embebido recibe la petición HTTP.
3. La infraestructura web de Spring procesa la petición.
4. El DispatcherServlet participa como *front controller* de Spring MVC.
5. El sistema de mappings encuentra un método compatible con GET /hola.
6. Spring invoca SaludoController.saludar().
7. El método devuelve un String.
8. La infraestructura MVC escribe esa representación en la respuesta.
9. Tomcat envía la respuesta HTTP al cliente.
10. curl muestra cabeceras y cuerpo.

Relación con Spring Boot

Tú no escribiste código para:

- crear un socket servidor;
- instanciar y configurar Tomcat manualmente;
- registrar a mano un DispatcherServlet;
- construir desde cero el pipeline MVC;
- coordinar todas las versiones de esas librerías.

Eso no significa que esas piezas no existan. Significa que Spring Boot y el ecosistema Spring las han preparado a partir del proyecto y sus convenciones.

Pregunta

¿Por qué este endpoint no demuestra todavía de forma suficiente el comportamiento de Jackson?

Respuesta razonada

Porque devuelve un String simple. Jackson está disponible en el proyecto gracias al stack web, pero una respuesta de texto no obliga a demostrar la serialización de un objeto Java complejo a JSON. En el paso 10 crearemos un probe temporal que sí lo haga.

Paso 8 - Explicar el papel de Spring Boot usando evidencias del proyecto

Sin modificar archivos, completa este mapa con evidencias que ya has observado:

Idea	Evidencia del proyecto
Auto-configuración	--debug y Conditions Evaluation Report
Starters	spring-boot-starter-web en pom.xml
Dependencias transitivas	dependency:tree muestra MVC, Jackson, Tomcat, etc.
Servidor embebido	JAR ejecutable + Tomcat en dependencias + puerto HTTP
Configuración flexible	server.port=9090 cambia el puerto
Component scan	SaludoController bajo el paquete raíz es descubierto
Runtime observable	/hola, /adios, logs y códigos HTTP

Ahora intenta explicar el proyecto en un párrafo sin utilizar la frase “Spring Boot hace magia”.

Una respuesta técnicamente útil debería parecerse conceptualmente a:

El POM declara una capacidad web mediante un starter. Maven resuelve sus dependencias. Spring Boot detecta el entorno web y activa configuraciones apropiadas. El servidor embebido arranca dentro del proceso y Spring MVC descubre el controlador. Las propiedades permiten sobrescribir valores por defecto.

No memorices esa frase; utiliza tus propias palabras.

Pregunta

¿Qué parte de esta cadena fallaría primero si Maven no hubiera descargado correctamente el starter web?

Respuesta razonada

El problema aparecería ya en el classpath: clases necesarias no estarían disponibles. Podríamos ver errores de compilación/importación o, según el cambio, dejarían de cumplirse condiciones de auto-configuración. No tendría sentido empezar diagnosticando el puerto antes de confirmar las dependencias.

Paso 9 - Diagnosticar errores frecuentes sin cambiar varias cosas a la vez

Utiliza esta tabla como punto de partida:

Síntoma	Hipótesis probable	Primera comprobación útil
No aparece el informe de condiciones Port 8080 already in use	- debug no llegó como argumento de programa otro proceso ocupa el puerto	revisar configuración de ejecución/comando identificar proceso o usar temporalmente otro puerto
/hola devuelve 404	controlador no registrado o ruta incorrecta	revisar mapping, paquetes y logs
El IDE no reconoce imports Spring La app termina al arrancar	Maven no cargó correctamente dependencias fallo de contexto o no es el tipo de aplicación esperado	ejecutar Wrapper / recargar Maven leer el primer error real del log
Una auto-configuración no aparece como positiva dependency:tree no muestra una pieza esperada	no se cumple alguna condición dependencia ausente/excluida	buscarla en Negative matches revisar POM y árbol
9090 sigue activo después del ejercicio	propiedad temporal no restaurada o proceso viejo	revisar application.properties y procesos

Método de diagnóstico

1. Reproduce el fallo.
2. Conserva el mensaje exacto.
3. Identifica la capa: Maven, arranque, configuración, MVC, HTTP.
4. Cambia una sola variable.
5. Repite la misma prueba.
6. Cuando resuelvas el fallo, restaura cualquier experimento que no forme parte del diseño final.

Pregunta

¿Por qué “borrar cachés y reiniciar todo” no debería ser nuestra primera estrategia?

Respuesta razonada

Porque puede hacer desaparecer evidencia y no enseña qué causó el problema. Algunas veces reiniciar o limpiar es útil, pero primero debemos intentar entender el síntoma y aislar la capa responsable.

Paso 10 - Reto resuelto: excluir temporalmente Jackson y demostrar el efecto

El problema del reto original

Nuestro endpoint /hola devuelve texto. Si excluimos Jackson y sólo volvemos a probar /hola, podríamos concluir erróneamente que “no ha pasado nada”. Para demostrar el efecto necesitamos un endpoint que obligue a representar un objeto estructurado como JSON.

Vamos a crear ese probe **sólo durante el experimento** y después restauraremos ambos archivos.

Estado inicial que debes conservar

Antes de editar, ejecuta:

```
./mvnw test
```

Debe terminar con BUILD SUCCESS.

Guarda mentalmente o mediante tu control de versiones el estado actual de:

```
src/main/java/es/meecd/demo/miproyecto/MiProyectoApplication.java  
src/main/java/es/meecd/demo/miproyecto/controller/SaludoController.java
```

10.1 Añadir un probe JSON temporal

En SaludoController.java, añade temporalmente estos imports:

```
import java.util.LinkedHashMap;  
import java.util.Map;
```

Y añade este método:

```
@GetMapping("/info-json")  
public Map<String, Object> infoJson() {  
    Map<String, Object> info = new LinkedHashMap<>();  
    info.put("aplicacion", "mi-proyecto");  
    info.put("modulo", 1);  
    info.put("jackson", true);  
    return info;  
}
```

Arranca la aplicación **sin excluir todavía Jackson** y ejecuta:

```
curl -i http://localhost:8080/info-json
```

Debes obtener una respuesta 200 con Content-Type JSON y un cuerpo estructurado equivalente a:

```
{"aplicacion":"mi-proyecto","modulo":1,"jackson":true}
```

El orden exacto queda estable en este ejemplo por usar LinkedHashMap, pero lo importante es que un objeto Java se convierte en JSON.

10.2 Excluir temporalmente la auto-configuración de Jackson

Detén la aplicación.

Cambia temporalmente la anotación de `MiProyectoApplication` a:

```
@SpringBootApplication(  
    exclude = org.springframework.boot.autoconfigure.jackson.JacksonAutoConfiguration.class)
```

No modifiques el método `main`.

Arranca de nuevo con `--debug` para que además puedas observar la exclusión:

```
./mvnw spring-boot:run -Dspring-boot.run.arguments=--debug
```

Busca `JacksonAutoConfiguration` en el informe y comprueba que aparece relacionada con las exclusiones.

Ahora repite:

```
curl -i http://localhost:8080/info-json
```

El objetivo no es memorizar un texto de excepción concreto. Lo que debes demostrar es que el endpoint que antes se representaba como JSON **ya no mantiene el mismo camino de serialización automática proporcionado por la configuración normal**. Observa conjuntamente:

- el código HTTP;
- el Content-Type si existe;
- el cuerpo;
- y, sobre todo, el error relevante de los logs si la representación falla.

La respuesta exacta puede variar con la versión y con otros conversores presentes en el classpath. La evidencia principal es la diferencia reproducible antes/después y la exclusión visible en el informe de condiciones.

10.3 Comprobar que /hola no es una prueba suficiente de Jackson

Ejecuta también:

```
curl -i http://localhost:8080/hola
```

Puede seguir respondiendo correctamente porque un String de texto tiene un tratamiento diferente a un objeto que necesita serialización JSON. Esta comparación es parte del aprendizaje: **una prueba debe ejercitar realmente la pieza que afirma validar**.

10.4 Restaurar MiProyectoApplication

Devuelve la anotación a:

```
@SpringBootApplication
```

10.5 Eliminar el probe temporal

Elimina de `SaludoController`:

- `LinkedHashMap`;
- `Map`;
- el método `infoJson()`;
- el mapping `/info-json`.

El controlador debe volver al estado heredado de M0.

10.6 Verificación de restauración

Ejecuta:

```
./mvnw test
```

Después arranca normalmente:

```
./mvnw spring-boot:run
```

Comprueba:

```
curl -i http://localhost:8080/hola
curl -i http://localhost:8080/adios
curl -i http://localhost:8080/info-json
```

Resultados esperados:

- /hola -> 200;
- /adios -> 200;
- /info-json -> 404, porque el probe ya no pertenece al snapshot final.

Finalmente revisa `application.properties` y confirma que tampoco conserva `server.port=9090`.

Qué has demostrado

Has hecho un experimento controlado completo:

baseline verde

- > crear observable específico
- > medir comportamiento normal
- > cambiar una sola pieza de auto-configuración
- > medir diferencia
- > restaurar
- > volver a validar baseline

Ese patrón es más valioso que memorizar el nombre de una clase concreta: es una forma general de investigar frameworks.

Pregunta

¿Por qué eliminamos `/info-json` aunque sería un endpoint útil para seguir experimentando?

Respuesta razonada

Porque su única función en este punto era aislar el papel de Jackson. El curso introducirá JSON y DTOs de forma permanente en 1.3. Mantener un endpoint auxiliar antes de enseñarlo formalmente contaminaría la progresión pedagógica y haría que el snapshot final contuviera comportamiento que no pertenece todavía a su diseño.

Resultado esperado global de la práctica 1.1

Al terminar debes poder comprobar simultáneamente:

- `./mvnw test` pasa;
- la aplicación arranca en 8080;
- `GET /hola` devuelve 200;
- `GET /adios` devuelve 200;
- `GET /info-json` devuelve 404 porque el probe temporal fue retirado;
- `application.properties` no contiene `server.port=9090`;
- `MiProyectoApplication` vuelve a usar `@SpringBootApplication` sin exclusiones;
- `SaludoController` vuelve a su estado heredado;

- `dependency:tree` demuestra que Tomcat, Spring Web/MVC y Jackson llegan a través del stack web;
- puedes localizar y explicar el *Conditions Evaluation Report*;
- puedes explicar con tus propias palabras la diferencia entre Spring Framework y Spring Boot, y el papel de auto-configuración, starters y servidor embebido.

Checklist de cierre

Antes de pasar a 1.2 responde sí a todo:

- ☐ ¿He arrancado el proyecto desde el baseline M0?
- ☐ ¿He visto el informe de condiciones con `--debug`?
- ☐ ¿Sé explicar un positive match y un negative match sin tratarlos como “éxito/error” automáticamente?
- ☐ ¿He demostrado Tomcat mediante el árbol de dependencias y el JAR ejecutable?
- ☐ ¿He cambiado temporalmente el puerto y lo he restaurado?
- ☐ ¿He inspeccionado las dependencias transitivas del starter web?
- ☐ ¿Puedo reconstruir la cadena cliente -> Tomcat -> Spring MVC -> controlador -> respuesta?
- ☐ ¿He probado la exclusión temporal de Jackson con un observable que realmente necesita JSON?
- ☐ ¿He restaurado completamente los dos archivos modificados en el reto?
- ☐ ¿Los tests y endpoints heredados vuelven a estar verdes?

En la práctica 1.2 utilizaremos este mismo proyecto para observar HTTP de forma explícita: petición, respuesta, cabeceras, métodos y códigos de estado. Allí el objetivo dejará de ser “qué configura Spring Boot” y pasará a ser “qué ocurre exactamente entre un cliente y este servidor”.

Práctica 1.2 - Observar HTTP con navegador, DevTools y curl

Objetivo

En esta práctica no construiremos todavía una API nueva. Utilizaremos el proyecto que ya funciona para observar **peticiones y respuestas HTTP reales** y aprender a distinguir conexión, ruta, método, cabeceras, cuerpo y código de estado.

Al finalizar volveremos a dejar M1/proyecto exactamente como estaba al empezar. El único código que añadiremos será un endpoint `/eco` temporal para provocar de forma reproducible un 400 Bad Request con JSON mal formado.

En Windows PowerShell utiliza `curl.exe` si `curl` está asociado a otro comando o alias. Los ejemplos se escriben con `curl` porque funcionan directamente en Linux/macOS y en instalaciones modernas de Windows con el ejecutable disponible.

Paso 1 - Arrancar la aplicación y formular hipótesis

Qué hacemos

Desde M1/proyecto, arranca la aplicación:

Linux/macOS:

```
./mvnw spring-boot:run
```

Windows:

```
mvnw.cmd spring-boot:run
```

Comprueba primero el baseline:

```
curl -i http://localhost:8080/hola
```

Debe responder 200 y mostrar:

Hola, Ministerio de Educación

Antes de ejecutar los siguientes pasos, escribe tus predicciones para estas dos preguntas:

1. ¿Qué estado esperas al pedir GET /hola?
2. ¿Qué estado esperas al pedir GET /no-existe?

Por qué empezamos así

Observar es más útil cuando antes hemos formulado una hipótesis. Si simplemente ejecutamos comandos y copiamos resultados, aprendemos menos que si anticipamos qué debería ocurrir y contrastamos la predicción.

Pregunta

¿Qué diferencia hay entre que la aplicación no esté arrancada y que la ruta solicitada no exista?

Respuesta razonada

Si la aplicación no está escuchando en el destino, el cliente puede obtener un error de conexión y no llega a existir una respuesta HTTP. Si la aplicación está viva pero no tiene un handler para la ruta, el servidor sí recibe la petición y puede responder con un estado HTTP como 404 Not Found.

Paso 2 - Probar GET con el navegador

Abre:

`http://localhost:8080/hola`

Debes ver el saludo heredado de M0.

Reconstruye lo que ha ocurrido

Sin entrar todavía en detalles de bajo nivel innecesarios, el recorrido es:

1. el navegador interpreta la URL;
2. resuelve localhost hacia la propia máquina;
3. abre una conexión al puerto 8080;
4. envía una petición HTTP GET /hola;
5. Tomcat embebido recibe la petición;
6. Spring MVC encuentra el mapping compatible;
7. se ejecuta `SaludoController.saludar()`;
8. el valor devuelto se convierte en el cuerpo de la respuesta;
9. el servidor envía la respuesta;
10. el navegador la presenta.

No fijas como requisito que localhost deba resolverse siempre exactamente a 127.0.0.1: según el sistema también puede intervenir IPv6 (: :1).

Pregunta

¿El navegador “llama” directamente al método `saludar()`?

Respuesta razonada

No. El navegador sólo conoce HTTP: método, host, puerto y ruta. Es la infraestructura del servidor la que transforma esa petición en la invocación de un handler Java. Esta separación permite que el cliente ignore por completo cómo está organizado internamente el back-end.

Paso 3 - Probar una ruta que no existe

Visita en el navegador:

`http://localhost:8080/no-existe`

La interfaz exacta del error puede variar según la versión y la configuración. No bases el aprendizaje en que aparezca necesariamente una página concreta o un JSON con campos fijos.

Lo estable que debes comprobar es el **estado 404**.

Si el navegador no lo muestra claramente, lo confirmaremos en el siguiente paso con curl y después en DevTools.

Qué significa

El servidor está vivo y ha recibido una petición, pero no existe ningún mapping que atienda GET /no-existe.

Pregunta

¿Un 404 demuestra que Spring Boot no está arrancado?

Respuesta razonada

No; de hecho demuestra lo contrario: alguien ha recibido la petición y ha producido una respuesta HTTP 404. Cuando no hay servidor accesible, normalmente obtenemos un error de conexión, no un código HTTP generado por la aplicación.

Paso 4 - Probar GET con curl y ver las cabeceras

Ejecuta primero:

`curl http://localhost:8080/hola`

Verás sólo el cuerpo.

Ahora añade -i:

`curl -i http://localhost:8080/hola`

Una respuesta real tendrá una estructura semejante a:

```
HTTP/1.1 200
Content-Type: text/plain;charset=UTF-8
Content-Length: ...
Date: ...
```

Hola, Ministerio de Educación

No memorices Content-Length ni Date: pueden cambiar. Comprueba estas propiedades estables:

- el estado es 200;
- el cuerpo contiene el saludo esperado;

- el servidor identifica el cuerpo como texto mediante Content-Type.

Pregunta

¿Qué añade curl -i respecto de curl sin opciones?

Respuesta razonada

Hace visibles las cabeceras de la respuesta además del cuerpo. Esto permite comprobar el código de estado y metadatos como Content-Type, que el navegador normalmente no presenta en la página renderizada.

Paso 5 - Probar una ruta inexistente con curl

Ejecuta:

```
curl -i http://localhost:8080/no-existe
```

Busca la primera línea de la respuesta. Debe indicar 404.

Dependiendo de la versión/configuración, Spring Boot puede devolver un cuerpo de error con distinta forma o incluso diferente nivel de detalle. No fijamos como contrato didáctico un JSON exacto porque ese detalle no es necesario para aprender la semántica HTTP y puede evolucionar.

Para obtener sólo el código de estado de forma reproducible:

Linux/macOS:

```
curl --silent --output /dev/null --write-out "%{http_code}\n" http://localhost:8080/no-existe
```

En PowerShell puedes usar:

```
curl.exe --silent --output NUL --write-out "%{http_code}`n" http://localhost:8080/no-existe
```

Pregunta

¿Por qué el código 404 es una evidencia más estable que copiar literalmente el cuerpo de error de una versión concreta?

Respuesta razonada

Porque 404 Not Found forma parte del contrato estándar HTTP que queremos observar. El formato detallado del error lo decide la aplicación/framework y puede cambiar entre versiones o por configuración sin que cambie el hecho semántico de que el recurso no existe.

Paso 6 - Probar un método HTTP no soportado

Envía ahora un método distinto contra una ruta existente:

```
curl -i -X POST http://localhost:8080/hola
```

Esperamos 405 Method Not Allowed.

Compara:

```
GET /no-existe -> 404
```

```
POST /hola -> 405
```

En el primer caso no existe un handler para la ruta solicitada. En el segundo sí existe /hola, pero nuestro controlador sólo declara:

```
@GetMapping("/hola")
```

Pregunta

¿Por qué Spring no trata POST /hola como si fuera simplemente GET /hola?

Respuesta razonada

Porque el método HTTP forma parte del contrato del endpoint. GET y POST expresan intenciones distintas y Spring MVC selecciona handlers usando, entre otros criterios, la ruta y el método. Ignorar esa diferencia haría imposible diseñar operaciones predecibles.

Paso 7 - Ver las cabeceras que envía el cliente con curl

Ejecuta:

```
curl -v http://localhost:8080/hola
```

curl escribe información de diagnóstico adicional. Verás líneas conceptualmente equivalentes a:

```
> GET /hola HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/...
> Accept: */*
>
< HTTP/1.1 200
< Content-Type: text/plain; charset=UTF-8
< ...
```

Hola, Ministerio de Educación

Las líneas prefijadas con > representan datos que envía el cliente y las que empiezan con < representan datos recibidos.

Observa particularmente:

- método y ruta;
- Host;
- User-Agent;
- Accept;
- estado de la respuesta;
- Content-Type.

Pregunta

¿User-Agent decide qué método Java ejecuta Spring MVC en nuestro ejemplo?

Respuesta razonada

No. Puede ser una cabecera útil para identificar al cliente, pero nuestro mapping de /hola se selecciona esencialmente por la combinación de ruta y método HTTP declarada. No hemos configurado una condición de mapping basada en User-Agent.

Paso 8 - Observar la misma petición con DevTools

Abre el navegador, pulsa F12 y entra en **Network** (Red).

Recarga:

http://localhost:8080/hola

Selecciona la petición correspondiente. Los nombres exactos de las pestañas pueden variar entre Chrome, Edge, Firefox y versiones, pero debes poder localizar:

- URL;
- método GET;
- estado 200;
- cabeceras de petición;
- cabeceras de respuesta;
- cuerpo/Response;
- tiempos o Timing.

Repite con:

http://localhost:8080/no-existe

Y compara el estado.

Cuatro entornos

Este paso ocurre en el navegador y es independiente de que hayas arrancado el servidor desde IntelliJ IDEA, Eclipse, VS Code o consola. Lo importante es distinguir **la herramienta que ejecuta el servidor** de **la herramienta que actúa como cliente/observador**.

Pregunta

¿DevTools y curl -v están observando protocolos diferentes?

Respuesta razonada

No. Ambos permiten observar la comunicación HTTP. La diferencia es la interfaz y el tipo de cliente: DevTools inspecciona lo que realiza el navegador, mientras que curl permite construir peticiones explícitas y reproducibles desde terminal.

Paso 9 - Razonar sobre lo observado

Sin modificar código, completa esta tabla con evidencias de tus propias pruebas:

Caso	Método	Ruta	Estado esperado	Qué demuestra
saludo	GET	/hola	200	ruta y método válidos
inexistente	GET	/no-existe	404	no existe handler/recurso solicitado
método incorrecto	POST	/hola	405	ruta conocida pero método no permitido

Pregunta

1. ¿Qué cambia entre 404 y 405?
2. ¿Qué información aporta Content-Type?
3. ¿Qué diferencia hay entre cuerpo y cabeceras?
4. ¿Qué herramienta te permite repetir exactamente una petición desde terminal?

Respuesta razonada

1. En 404 no se encuentra el recurso/handler solicitado; en 405 la ruta se reconoce en un contexto donde ese método no está permitido.
2. Content-Type describe el tipo de representación del cuerpo.
3. El cuerpo transporta los datos principales de la representación; las cabeceras transportan meta-datos de la comunicación.
4. curl permite expresar la petición mediante un comando reproducible.

Paso 10 - Diagnosticar errores comunes de comunicación HTTP

Utiliza esta tabla como mapa de capas, no como una colección para memorizar:

Síntoma	Capa/causa probable	Comprobación inicial
Connection refused / no conecta	no hay proceso escuchando o puerto incorrecto	logs, proceso y puerto
404 Not Found	ruta/recurso no encontrado	URL y mappings
405 Method Not Allowed	método incompatible con la ruta	GET/POST/etc. declarado
400 Bad Request	petición/cuerpo no interpretable	sintaxis y logs de conversión
415 Unsupported Media Type	tipo de contenido no soportado/ausente para esa operación	Content-Type
500 Internal Server Error	fallo inesperado en procesamiento servidor	primer error relevante de logs
curl: command not found	herramienta no disponible	instalar/usar curl.exe, navegador u otro cliente
respuesta distinta de la guía	versión/configuración produce detalles distintos	comprobar primero estado y contrato estable

Regla de diagnóstico

No cambies código Java si el cliente ni siquiera puede establecer conexión. No busques un error de JSON si estás obteniendo 404 en otra ruta. Empieza por la capa más baja que pueda explicar el síntoma.

Pregunta

Si curl devuelve Connection refused, ¿tiene sentido buscar un @GetMapping incorrecto como primera hipótesis?

Respuesta razonada

No. Un mapping incorrecto sólo importa después de que la petición haya llegado al servidor. Connection refused indica normalmente que el problema ocurre antes: proceso, dirección o puerto.

Paso 11 - Resumir lo observado con pruebas reproducibles

Ejecuta este pequeño conjunto final:

```
curl -i http://localhost:8080/hola
curl -i http://localhost:8080/no-existe
curl -i -X POST http://localhost:8080/hola
curl -v http://localhost:8080/hola
```

Debes ser capaz de explicar, sin mirar la teoría:

- por qué el primero es 200;
- por qué el segundo es 404;
- por qué el tercero es 405;
- dónde ves cabeceras enviadas y recibidas en el cuarto;
- por qué Content-Type forma parte de la respuesta;
- por qué todos estos resultados proceden del mismo servidor y, sin embargo, representan situaciones diferentes.

Pregunta

¿Cuál de estas pruebas modifica de forma permanente el proyecto?

Respuesta razonada

Ninguna. Todas son observaciones externas del servidor existente. En el paso 12 sí añadiremos temporalmente un endpoint para crear una petición que requiera interpretar JSON, y lo eliminaremos al terminar.

Paso 12 - Reto resuelto: provocar un 400 Bad Request y restaurar el proyecto

Objetivo

Queremos provocar un 400 Bad Request debido a **JSON mal formado**, no fabricar un 400 manualmente. Para ello la petición debe llegar a un endpoint cuyo cuerpo tenga que ser deserializado por Jackson.

12.1 Baseline

Antes de tocar código:

```
./mvnw test
```

Debe terminar correctamente.

Comprueba también:

```
curl -i http://localhost:8080/hola
```

12.2 Añadir temporalmente /eco

Detén la aplicación.

En SaludoController.java, añade temporalmente:

```
import org.springframework.web.bind.annotation.PostMapping;  
import org.springframework.web.bind.annotation.RequestBody;
```

```
import java.util.Map;
```

Y dentro de la clase:

```
@PostMapping("/eco")  
public Map<String, Object> eco(@RequestBody Map<String, Object> mensaje) {  
    return mensaje;  
}
```

¿Por qué utilizamos Map<String, Object> y no simplemente String? Porque necesitamos obligar a Jackson a **interpretar** el cuerpo como JSON estructurado. Si recibiéramos un String crudo, un texto con sintaxis JSON inválida podría llegar al método sin que Jackson tuviera que parsearlo como objeto.

12.3 Probar JSON mal formado

Arranca la aplicación y ejecuta:

```
curl -i -X POST http://localhost:8080/eco \
-H "Content-Type: application/json" \
-d '{"mensaje":"hola",}'
```

La coma final hace que el JSON sea inválido.

Debes comprobar **dos evidencias**:

1. el estado HTTP es 400;
2. los logs muestran un error de lectura/conversión JSON coherente con la petición mal formada.

No fijamos como contrato el cuerpo de error completo ni un mensaje textual exacto: esos detalles pueden variar por versión/configuración.

12.4 Probar JSON válido

Ejecuta:

```
curl -i -X POST http://localhost:8080/eco \
-H "Content-Type: application/json" \
-d '{"mensaje":"hola"}'
```

Ahora esperamos 200 y un cuerpo JSON equivalente a:

```
{"mensaje":"hola"}
```

Has cambiado una sola variable: la sintaxis del cuerpo.

12.5 Diferenciar 400 de 415

Como observación adicional, prueba sin declarar el tipo correcto:

```
curl -i -X POST http://localhost:8080/eco \
-H "Content-Type: text/plain" \
-d '{"mensaje":"hola"}'
```

La respuesta puede ser 415 Unsupported Media Type porque el endpoint espera una representación que Spring pueda convertir a Map desde el tipo recibido. Esta prueba refuerza que **formato inválido** y **tipo de medio no soportado** son problemas distintos.

12.6 Restauración obligatoria

Detén la aplicación y elimina de SaludoController.java:

- los imports PostMapping, RequestBody y Map añadidos para el reto;
- el método eco();
- el mapping /eco.

El archivo debe volver exactamente al estado heredado.

Ejecuta:

```
./mvnw test
```

Arranca normalmente y verifica:

```
curl -i http://localhost:8080/hola
curl -i http://localhost:8080/adios
curl -i -X POST http://localhost:8080/eco
```

Resultados finales:

- /hola -> 200;

- /adios -> 200;
- /eco ya no forma parte de la aplicación; una petición POST a esa ruta no debe comportarse como el endpoint temporal.

Pregunta

¿Por qué este reto usa un endpoint temporal si 1.2 pretende principalmente observar HTTP?

Respuesta razonada

Porque algunos estados HTTP sólo pueden comprenderse bien si provocamos una condición concreta. El endpoint temporal crea una superficie mínima para observar cómo un cuerpo JSON inválido produce un error de petición. Después se elimina para no adelantar permanentemente el contenido de JSON/Jackson que se estudiará formalmente en 1.3.

Resultado esperado global de la práctica 1.2

Al terminar debes poder demostrar que:

- distingues cliente y servidor aunque estén en localhost;
- entiendes el ciclo petición-respuesta;
- puedes identificar método, ruta, cabeceras, estado y cuerpo;
- GET /hola devuelve 200;
- GET /no-existe devuelve 404;
- POST /hola devuelve 405;
- sabes observar la conversación mediante `curl -i`, `curl -v` y DevTools;
- puedes diferenciar error de conexión de un estado HTTP;
- has provocado de forma controlada un 400 con JSON inválido;
- has comprobado que un JSON válido funciona en el mismo endpoint temporal;
- has eliminado /eco y sus imports;
- `./mvnw test` vuelve a estar verde;
- el snapshot final sigue sin cambios funcionales permanentes respecto del inicio de 1.2.

Checklist de cierre

- ☐ ¿La app responde en 8080?
 - ☐ ¿He comprobado 200, 404 y 405 con peticiones distintas?
 - ☐ ¿He observado cabeceras de petición y respuesta?
 - ☐ ¿He usado DevTools Network?
 - ☐ ¿Sé diferenciar cabeceras y cuerpo?
 - ☐ ¿Sé diferenciar 404 y 405?
 - ☐ ¿He provocado 400 mediante JSON realmente mal formado?
 - ☐ ¿He comprobado el caso JSON válido?
 - ☐ ¿He eliminado /eco y los imports temporales?
 - ☐ ¿Los tests heredados vuelven a pasar?
-

En la práctica 1.3 dejaremos de usar JSON sólo como herramienta de diagnóstico y lo convertiremos en parte permanente del contrato de nuestra aplicación mediante DTOs y Jackson.

Práctica 1.3 - Jackson y DTOs

Objetivo

Hasta ahora JSON apareció como representación observada desde fuera. En esta práctica pasará a formar parte **permanente** del proyecto: crearemos un DTO, un controlador que lo devuelve y recibe,

y personalizaremos progresivamente su contrato JSON.

Al contrario que 1.1 y 1.2, el estado final de M1/proyecto **sí evolucionará**. Al terminar existirán permanentemente:

```
src/main/java/es/mecd/demo/miproyecto/  
|-- controller/  
|   |-- SaludoController.java  
|   |-- ExpedienteController.java  
|-- dto/  
|   |-- ExpedienteDTO.java  
|   |-- SolicitanteDTO.java
```

No añadiremos todavía persistencia, servicios ni validación Bean Validation. El objetivo es aislar la representación y la conversión JSON.

Paso 1 - Crear el paquete dto

Qué hacemos

Bajo el paquete raíz:

`es.mecd.demo.miproyecto`

crea:

`es.mecd.demo.miproyecto.dto`

Ruta física:

`src/main/java/es/mecd/demo/miproyecto/dto/`

Consola

Linux/macOS:

```
mkdir -p src/main/java/es/mecd/demo/miproyecto/dto
```

PowerShell:

```
New-Item -ItemType Directory -Force src/main/java/es/mecd/demo/miproyecto/dto
```

IntelliJ IDEA

Clic derecho sobre `es.mecd.demo.miproyecto` → **New > Package** → `dto`.

Eclipse

Clic derecho sobre `es.mecd.demo.miproyecto` → **New > Package** → completa el nombre con `.dto`.

VS Code

Crea la carpeta `dto` bajo el paquete raíz y asegúrate de que el archivo Java que añadiremos tenga:

```
package es.mecd.demo.miproyecto.dto;
```

Por qué un paquete específico

El DTO representa datos que cruzan el contrato de la API. Separarlo del controlador evita mezclar estructura de representación con lógica de entrada HTTP.

Pregunta

¿Por qué no colocamos ExpedienteDTO dentro del paquete controller si el controlador será quien lo devuelva?

Respuesta razonada

Porque el DTO no es un controlador ni debería depender de cómo llega la petición. Es una representación de datos reutilizable por controladores y, más adelante, servicios. Separar paquetes hace visible esa responsabilidad.

Paso 2 - Crear la clase ExpedienteDTO

Crea:

src/main/java/es/mecd/demo/miproyecto/dto/ExpedienteDTO.java

En este primer estado **todavía no añadimos anotaciones de Jackson**:

```
package es.mecd.demo.miproyecto.dto;

import java.time.LocalDate;
import java.util.List;

public class ExpedienteDTO {

    private String identificador;
    private String titular;
    private String dni;
    private String estado;
    private String tipo;
    private LocalDate fechaSolicitud;
    private Double importe;
    private Boolean activo;
    private List<String> documentos;
    private String observaciones;

    public ExpedienteDTO() {
    }

    public ExpedienteDTO(
        String identificador,
        String titular,
        String dni,
        String estado,
        String tipo,
        LocalDate fechaSolicitud,
        Double importe,
        Boolean activo,
        List<String> documentos) {
        this.identificador = identificador;
        this.titular = titular;
        this.dni = dni;
        this.estado = estado;
        this.tipo = tipo;
        this.fechaSolicitud = fechaSolicitud;
        this.importe = importe;
        this.activo = activo;
    }
}
```

```
        this.documentos = documentos;
    }

    public String getIdentificador() {
        return identificador;
    }

    public void setIdentificador(String identificador) {
        this.identificador = identificador;
    }

    public String getTitular() {
        return titular;
    }

    public void setTitular(String titular) {
        this.titular = titular;
    }

    public String getDni() {
        return dni;
    }

    public void setDni(String dni) {
        this.dni = dni;
    }

    public String getEstado() {
        return estado;
    }

    public void setEstado(String estado) {
        this.estado = estado;
    }

    public String getTipo() {
        return tipo;
    }

    public void setTipo(String tipo) {
        this.tipo = tipo;
    }

    public LocalDate getFechaSolicitud() {
        return fechaSolicitud;
    }

    public void setFechaSolicitud(LocalDate fechaSolicitud) {
        this.fechaSolicitud = fechaSolicitud;
    }

    public Double getImporte() {
        return importe;
    }

    public void setImporte(Double importe) {
        this.importe = importe;
    }
}
```

```
}

public Boolean getActivo() {
    return activo;
}

public void setActivo(Boolean activo) {
    this.activo = activo;
}

public List<String> getDocumentos() {
    return documentos;
}

public void setDocumentos(List<String> documentos) {
    this.documentos = documentos;
}

public String getObservaciones() {
    return observaciones;
}

public void setObservaciones(String observaciones) {
    this.observaciones = observaciones;
}
}
```

Ejecuta:

```
./mvnw test
```

La clase debe compilar antes de utilizarla en HTTP.

Qué debes observar

Tenemos varios tipos Java para provocar conversiones diferentes:

- String;
- LocalDate;
- Double;
- Boolean;
- List<String>;
- un campo observaciones que dejaremos nulo inicialmente.

El constructor sin argumentos y los accesoros mantienen un DTO JavaBean sencillo y fácil de deserializar.

Pregunta

¿Por qué incluimos constructor sin argumentos si ya tenemos otro constructor cómodo con datos?

Respuesta razonada

Porque queremos una clase que Jackson pueda instanciar y rellenar de forma convencional al deserializar. Jackson admite otras estrategias, pero el patrón JavaBean deja visible el proceso y es suficiente para este punto.

Paso 3 - Crear un controlador para probar la serialización

Crea:

src/main/java/es/mecd/demo/miproyecto/controller/ExpedienteController.java

Contenido inicial:

```
package es.mecd.demo.miproyecto.controller;

import es.mecd.demo.miproyecto.dto.ExpedienteDTO;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.time.LocalDate;
import java.util.List;

@RestController
@RequestMapping("/api/v1/expedientes")
public class ExpedienteController {

    @GetMapping("/ejemplo")
    public ExpedienteDTO ejemplo() {
        return new ExpedienteDTO(
            "12345",
            "Ana García López",
            "12345678A",
            "EN_TRAMITE",
            "BECA",
            LocalDate.of(2025, 1, 15),
            1500.00,
            true,
            List.of("DNI.pdf", "Notas.pdf"));
    }
}
```

Arranca y comprueba que el contexto sigue cargando:

```
./mvnw spring-boot:run
```

Antes de llamar al endpoint

Predice:

- nombre de la propiedad del identificador;
- formato de la fecha;
- representación del booleano;
- representación de la lista;
- qué ocurrirá con observaciones.

Pregunta

¿Dónde aparece en este controlador una llamada explícita a ObjectMapper?

Respuesta razonada

En ningún sitio. @RestController hace que Spring MVC trate el valor devuelto como cuerpo de respuesta y seleccione un conversor. Con JSON disponible, la integración con Jackson serializa el DTO sin que el controlador invoque manualmente al ObjectMapper.

Paso 4 - Arrancar y probar la serialización por defecto

Ejecuta:

```
curl -i http://localhost:8080/api/v1/expedientes/ejemplo
```

Con nuestra baseline Spring Boot 3.5.16 debes observar un 200 y Content-Type: application/json.

El cuerpo será conceptualmente:

```
{
  "identificador": "12345",
  "titular": "Ana García López",
  "dni": "12345678A",
  "estado": "EN_TRAMITE",
  "tipo": "BECA",
  "fechaSolicitud": "2025-01-15",
  "importe": 1500.0,
  "activo": true,
  "documentos": ["DNI.pdf", "Notas.pdf"],
  "observaciones": null
}
```

El orden de propiedades no debe considerarse un contrato salvo que lo configuremos expresamente.

Corrección respecto de material antiguo

En configuraciones antiguas o distintas puedes encontrar ejemplos donde LocalDate aparece como array. En **nuestro proyecto actual**, Spring Boot registra soporte Java Time y la fecha ya se representa normalmente como ISO. En el paso 6 añadiremos @JsonFormat para hacer explícito el contrato, no para afirmar que antes estuviera necesariamente mal.

Pregunta

¿Qué parte de la salida demuestra que Jackson ha interpretado tipos y no ha llamado simplemente a toString() sobre todo el objeto?

Respuesta razonada

El resultado es un objeto JSON estructurado: el booleano aparece como true, el número como número, la lista como array y cada propiedad tiene su valor. No es una cadena Java opaca con el nombre de clase y hash.

Paso 5 - Renombrar el campo con @JsonProperty

Queremos que el contrato externo exponga id, aunque en Java conservemos identificador.

En ExpedienteDT0.java, añade:

```
import com.fasterxml.jackson.annotation.JsonProperty;
```

Y sobre el campo:

```
@JsonProperty("id")
private String identificador;
```

Reinicia y prueba:

```
curl http://localhost:8080/api/v1/expedientes/ejemplo
```

Ahora debe aparecer:

```
"id": "12345"
```

Y no una propiedad identificador separada.

Qué no ha cambiado

Dentro de Java seguimos utilizando:

```
getIdificador()  
setIdificador(...)
```

La anotación modifica la representación externa.

Pregunta

¿Por qué puede ser útil cambiar el nombre JSON sin cambiar inmediatamente todos los nombres internos Java?

Respuesta razonada

Porque contrato externo e implementación interna pueden evolucionar con ritmos distintos. @JsonProperty permite mantener un nombre Java descriptivo y exponer una convención de API estable. Hay que usarlo con criterio: demasiadas diferencias de nombres también dificultan el mantenimiento.

Paso 6 - Formatear la fecha con @JsonFormat

Añade:

```
import com.fasterxml.jackson.annotation.JsonFormat;
```

Y sobre fechaSolicitud:

```
@JsonFormat(pattern = "yyyy-MM-dd")  
private LocalDate fechaSolicitud;
```

Prueba:

```
curl http://localhost:8080/api/v1/expedientes/ejemplo
```

Debe mantenerse el contrato:

```
"fechaSolicitud": "2025-01-15"
```

Qué demuestra realmente este paso

No estamos suponiendo que antes existiera un array. Estamos declarando localmente la representación que queremos conservar.

También afectará a la deserialización del paso 9: una cadena compatible con el patrón podrá convertirse a LocalDate; una fecha con formato incompatible producirá un error de conversión.

Pregunta

¿Por qué tiene sentido anotar el formato aunque la salida por defecto actual ya coincida?

Respuesta razonada

Porque hace explícita una decisión contractual cerca del campo que la usa. El comportamiento deja de depender sólo de una configuración global implícita del ObjectMapper. Aun así, en APIs grandes conviene diseñar una política de fechas global y coherente, no anotar arbitrariamente cada campo con formatos distintos.

Paso 7 - Omitir campos nulos con @JsonInclude

En ExpedienteDTO.java añade:

```
import com.fasterxml.jackson.annotation.JsonInclude;
```

Sobre la clase:

```
@JsonInclude(JsonInclude.Include.NON_NULL)
```

```
public class ExpedienteDTO {
```

observaciones permanece null en /ejemplo. Vuelve a probar:

```
curl http://localhost:8080/api/v1/expedientes/ejemplo
```

Ahora observaciones **no debe aparecer**.

Comprueba que las propiedades con valor siguen presentes.

Pregunta

¿@JsonInclude(NON_NULL) elimina el campo de la clase Java?

Respuesta razonada

No. El campo sigue existiendo y puede tener getter, setter y valor. La anotación controla si una propiedad nula se incluye en la representación JSON generada; no modifica la estructura de la clase compilada.

Paso 8 - Excluir un campo con @JsonIgnore

Añade un dato que deliberadamente no queremos exponer:

```
import com.fasterxml.jackson.annotation.JsonIgnore;
```

En el DTO:

```
@JsonIgnore
```

```
private String numeroSeguridadSocial;
```

Añade también getter y setter:

```
public String getNumeroSeguridadSocial() {  
    return numeroSeguridadSocial;  
}
```

```
public void setNumeroSeguridadSocial(String numeroSeguridadSocial) {  
    this.numeroSeguridadSocial = numeroSeguridadSocial;  
}
```

Para demostrar que la ausencia se debe realmente a @JsonIgnore y no a NON_NULL, cambia ejemplo() para construir el DTO en una variable y asignar un valor:

```

ExpedienteDTO dto = new ExpedienteDTO(
    "12345",
    "Ana García López",
    "12345678A",
    "EN_TRAMITE",
    "BECA",
    LocalDate.of(2025, 1, 15),
    1500.00,
    true,
    List.of("DNI.pdf", "Notas.pdf"));

dto.setNumeroSeguridadSocial("12/34567890/12");
return dto;

```

Prueba:

```
curl http://localhost:8080/api/v1/expedientes/ejemplo
```

numeroSeguridadSocial no debe aparecer aunque su valor **no sea null**.

Pregunta

¿Por qué era importante asignar un valor antes de comprobar @JsonIgnore?

Respuesta razonada

Porque la clase ya tiene @JsonInclude(NON_NULL). Si dejáramos el campo a null, desaparecería incluso sin @JsonIgnore y la prueba no distinguiría qué mecanismo causó la ausencia. Un buen experimento aísla la variable que quiere demostrar.

Paso 9 - Probar la deserialización con POST

Hasta ahora recorrimos Java → JSON. Añadiremos el camino JSON → Java.

En ExpedienteController.java añade:

```

import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;

```

Y el endpoint:

```

@PostMapping("/eco")
public ExpedienteDTO eco(@RequestBody ExpedienteDTO dto) {
    return dto;
}

```

Prueba:

```

curl -i -X POST http://localhost:8080/api/v1/expedientes/eco \
-H "Content-Type: application/json" \
-d '{
  "id": "99999",
  "titular": "María López",
  "dni": "11111111C",
  "estado": "NUEVO",
  "tipo": "BECA",
  "fechaSolicitud": "2025-01-20",
  "importe": 1200.0,
  "activo": true,

```

```
"documentos":["DNI.pdf"]
}'
```

Esperamos 200 y una representación equivalente a la entrada.

Sigue el recorrido

1. Spring MVC lee Content-Type: application/json.
2. El conversor Jackson interpreta el cuerpo.
3. id se mapea a identificador gracias a @JsonProperty.
4. fechaSolicitud se convierte a LocalDate.
5. Jackson rellena el DTO.
6. El método recibe el objeto Java.
7. Al devolverlo, se serializa de nuevo.

Prueba también una fecha inválida

```
curl -i -X POST http://localhost:8080/api/v1/expedientes/eco \
-H "Content-Type: application/json" \
-d '{"id":"1","fechaSolicitud":"20/01/2025"}'
```

Con el patrón definido esperamos un 400 por fallo de conversión.

Pregunta

¿El método eco() recibe el texto JSON crudo?

Respuesta razonada

No. @RequestBody ExpedienteDTO solicita que Spring convierta el cuerpo a esa clase antes de invocar el método. Si la conversión falla, el método puede no llegar a ejecutarse.

Paso 10 - Probar la deserialización con campos desconocidos

Envía una propiedad que no existe en ExpedienteDTO:

```
curl -i -X POST http://localhost:8080/api/v1/expedientes/eco \
-H "Content-Type: application/json" \
-d '{
  "id":"99999",
  "titular":"María López",
  "campoInexistente":"valor"
}'
```

Con el ObjectMapper auto-configurado de nuestro proyecto esperamos que la petición se procese sin error y que campoInexistente no aparezca en la salida.

Precisión importante

No memorices “Jackson siempre ignora campos desconocidos”. La configuración de Spring Boot que utilizamos los tolera de esta manera. Un ObjectMapper configurado con fallo ante propiedades desconocidas podría responder de forma distinta.

Prueba @JsonIgnore también en entrada

Envía:

```
curl -i -X POST http://localhost:8080/api/v1/expedientes/eco \
-H "Content-Type: application/json" \
-d '{"id": "1", "numeroSeguridadSocial": "dato-que-no-debe-entrar"}'
```

La propiedad ignorada no debe formar parte del DTO serializado de vuelta.

Pregunta

¿Por qué tolerar campos desconocidos puede ayudar a la evolución de una API y qué riesgo tiene?

Respuesta razonada

Puede permitir que clientes y servidores de versiones cercanas convivan cuando aparecen propiedades nuevas. El riesgo es ocultar errores tipográficos del cliente: `titualr` podría ignorarse silenciosamente. Por eso la política debe elegirse conscientemente y acompañarse de validación y documentación.

Paso 11 - Errores comunes del ejercicio

Utiliza esta tabla para diagnosticar por síntoma:

Síntoma	Causa probable	Comprobación
400 con JSON mal formado	error de sintaxis	validar comillas, comas y llaves
400 con fecha	patrón incompatible	revisar yyyy-MM-dd y valor recibido
una propiedad sale con nombre inesperado	contrato por defecto/anotación incorrecta	revisar @JsonProperty
aparece un null que querías omitir	falta/posición de @JsonInclude	revisar anotación de clase/campo
aparece un dato sensible	falta/uso incorrecto de @JsonIgnore	asignar valor y comprobar respuesta
campo queda null al entrar	nombre/tipo/accesores no compatibles	revisar JSON y DTO
propiedad desconocida no falla	configuración tolerante de Spring Boot	no confundir con una ley universal de Jackson
415	tipo de medio no convertible	revisar Content-Type
404	ruta incorrecta	revisar /api/v1/expedientes/eco
405	método incorrecto	usar POST en /eco

Estrategia

1. comprueba primero estado HTTP;
2. lee el primer error útil del log;
3. reduce el JSON al mínimo que reproduce el problema;
4. compara nombre JSON ↔ propiedad Java;
5. comprueba tipo y formato;
6. cambia una sola cosa y repite.

Pregunta

¿Por qué un 400 por fecha inválida y un 404 deben investigarse en lugares diferentes?

Respuesta razonada

El 404 indica que no se encontró la operación solicitada, por lo que debemos revisar ruta/mapping. El 400 de conversión significa que el endpoint sí fue localizado pero el cuerpo no pudo convertirse correctamente. Son capas distintas del recorrido HTTP.

Paso 12 - Reto resuelto: DTO con DTO anidado

Queremos agrupar datos del solicitante en un objeto propio.

12.1 Crear SolicitanteDTO

Crea:

src/main/java/es/mecd/demo/miproyecto/dto/SolicitanteDTO.java

```
package es.mecd.demo.miproyecto.dto;

import com.fasterxml.jackson.annotation.JsonProperty;

public class SolicitanteDTO {

    private String nombre;
    private String apellidos;

    @JsonProperty("dni")
    private String documentoIdentidad;

    public SolicitanteDTO() {
    }

    public SolicitanteDTO(String nombre, String apellidos, String documentoIdentidad) {
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.documentoIdentidad = documentoIdentidad;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellidos() {
        return apellidos;
    }

    public void setApellidos(String apellidos) {
        this.apellidos = apellidos;
    }

    public String getDocumentoIdentidad() {
        return documentoIdentidad;
    }
}
```

```

    public void setDocumentoIdentidad(String documentoIdentidad) {
        this.documentoIdentidad = documentoIdentidad;
    }
}

```

12.2 Añadirlo a ExpedienteDTO

Añade:

```
private SolicitanteDTO solicitante;
```

Y:

```

public SolicitanteDTO getSolicitante() {
    return solicitante;
}

public void setSolicitante(SolicitanteDTO solicitante) {
    this.solicitante = solicitante;
}

```

12.3 Evolucionar /ejemplo

Importa:

```
import es.mecd.demo.miproyecto.dto.SolicitanteDTO;
```

Y deja el método de esta forma:

```

@GetMapping("/ejemplo")
public ExpedienteDTO ejemplo() {
    ExpedienteDTO dto = new ExpedienteDTO(
        "12345",
        "Ana García López",
        "12345678A",
        "EN_TRAMITE",
        "BECA",
        LocalDate.of(2025, 1, 15),
        1500.00,
        true,
        List.of("DNI.pdf", "Notas.pdf"));
    dto.setNumeroSeguridadSocial("12/34567890/12");
    dto.setSolicitante(new SolicitanteDTO(
        "Ana", "García López", "12345678A"));
    return dto;
}

```

12.4 Probar serialización anidada

curl http://localhost:8080/api/v1/expedientes/ejemplo

Entre otras propiedades debe aparecer:

```

{
  "id": "12345",
  "titular": "Ana García López",
  "solicitante": {
    "nombre": "Ana",
    "apellidos": "García López",
    "dni": "12345678A"
  }
}

```

Y debe seguir **sin** aparecer numeroSeguridadSocial.

12.5 Probar deserialización anidada

```
curl -i -X POST http://localhost:8080/api/v1/expedientes/eco \
-H "Content-Type: application/json" \
-d '{
  "id": "77",
  "titular": "Lucía Pérez",
  "solicitante": {
    "nombre": "Lucía",
    "apellidos": "Pérez Ruiz",
    "dni": "22222222D"
  }
}'
```

El objeto solicitante debe volver anidado en la respuesta.

Cierre técnico

Ejecuta:

```
./mvnw test
./mvnw -DskipTests package
```

Comprueba además que siguen funcionando los endpoints heredados:

```
curl -i http://localhost:8080/hola
curl -i http://localhost:8080/adios
```

Pregunta

¿Qué ventaja tiene anidar SolicitanteDTO frente a añadir nombreSolicitante, apellidosSolicitante, dniSolicitante y más campos planos al expediente?

Respuesta razonada

Agrupar datos que forman una unidad conceptual, reduce prefijos repetidos y permite reutilizar/evolucionar la estructura del solicitante de forma más clara. El anidamiento también hace explícita esa relación en el JSON.

Resultado esperado global de la práctica 1.3

Al finalizar deben permanecer en el snapshot:

- paquete dto;
- ExpedienteDTO con varios tipos y constructor/accesores;
- @JsonProperty("id");
- @JsonFormat(pattern = "yyyy-MM-dd");
- @JsonInclude(NON_NULL);
- @JsonIgnore sobre numeroSeguridadSocial;
- SolicitanteDTO anidado con documentoIdentidad expuesto como dni;
- ExpedienteController con:
 - GET /api/v1/expedientes/ejemplo;
 - POST /api/v1/expedientes/eco;
- serialización y deserialización verificadas con HTTP real;
- campos desconocidos tolerados por la configuración actual;
- fechas inválidas rechazadas con 400;

- comportamiento heredado /hola y /adios intacto;
- tests y empaquetado verdes.

Checklist de cierre

- ☐ ¿El endpoint de ejemplo devuelve application/json?
- ☐ ¿El identificador externo se llama id?
- ☐ ¿La fecha usa yyyy-MM-dd?
- ☐ ¿Los nulos se omiten?
- ☐ ¿El número de seguridad social no aparece aunque tenga valor?
- ☐ ¿POST /eco deserializa y vuelve a serializar el DTO?
- ☐ ¿Una fecha incompatible produce 400?
- ☐ ¿La configuración actual tolera una propiedad desconocida?
- ☐ ¿El solicitante aparece como objeto anidado?
- ☐ ¿/hola y /adios siguen respondiendo?
- ☐ ¿./mvnw test pasa?

En 1.4 reutilizaremos estos conceptos para diseñar el recurso Alumno: URLs, métodos, estados, versionado y filtros antes de completar su CRUD.

Práctica 1.4 - Diseño de APIs REST

Objetivo práctico

Diseñar y construir una primera API REST del recurso Alumno aplicando deliberadamente convenciones de recursos, URLs, métodos y códigos HTTP. El punto no implementa todavía el CRUD completo: deja una API con GET colección, GET individual, POST con 201 y filtro opcional por curso. El siguiente punto completará la persistencia en memoria y el resto de operaciones.

Estado de partida

Debes conservar todo lo cerrado en 1.1–1.3. En particular siguen funcionando:

```
GET /hola
GET /adios
GET /api/v1/expedientes/ejemplo
POST /api/v1/expedientes/eco
```

Y el proyecto contiene ExpedienteDTO, SolicitanteDTO y ExpedienteController.

Para trabajar usa la carpeta M1/proyecto.

Desde la carpeta M1/proyecto, en Linux/macOS:

```
./mvnw spring-boot:run
```

En Windows:

```
mvnw.cmd spring-boot:run
```

Desde IntelliJ IDEA

Abre MiProyectoApplication.java y ejecuta Run 'MiProyectoApplication'. Cuando modifiques clases, vuelve a ejecutar si el cambio no se recarga automáticamente.

Desde Eclipse

Importa el proyecto como Maven y ejecuta MiProyectoApplication con Run As > Java Application.

Desde VS Code

Abre la carpeta que contiene pom.xml, asegúrate de usar JDK 17 y ejecuta la clase principal desde el soporte Java/Spring o desde el terminal integrado.

Paso 1 - Identificar los recursos y sus operaciones

Antes de escribir código, diseña el contrato. Para el dominio de alumnos queremos reconocer estas identidades:

```
/api/v1/alumnos
/api/v1/alumnos/{id}
/api/v1/alumnos/{id}/documentos
```

Y estas operaciones deseadas a medio plazo:

Operación	Método	URL
listar alumnos	GET	/api/v1/alumnos
consultar alumno	GET	/api/v1/alumnos/{id}
crear alumno	POST	/api/v1/alumnos
actualizar alumno	PUT	/api/v1/alumnos/{id}
eliminar alumno	DELETE	/api/v1/alumnos/{id}
listar documentos	GET	/api/v1/alumnos/{id}/documentos

No implementes todavía PUT, DELETE ni el subrecurso documentos. La tabla es **diseño del contrato**; en 1.4 materializaremos GET colección, POST, GET individual y filtro. El CRUD completo llegará en 1.5.

Qué debes observar

La misma URL de colección puede admitir GET y POST porque la operación la expresa el método HTTP. El ID individual forma parte de la identidad del recurso y, por tanto, aparece en el path.

Pregunta

¿Por qué el ID de un alumno va en /api/v1/alumnos/{id} y no en ?id=...?

Respuesta razonada

Porque el ID identifica un recurso concreto. Un query parameter modifica una consulta sobre un recurso o colección; no es la forma más clara de expresar la identidad principal. /alumnos/1 significa directamente “el recurso alumno 1”.

Paso 2 - Crear el DTO AlumnoDTO

El package dto ya existe desde 1.3. Crea:

M1/proyecto/src/main/java/es/mecd/demo/miproyecto/dto/AlumnoDTO.java

con este contenido:

```
package es.mecd.demo.miproyecto.dto;

import com.fasterxml.jackson.annotation.JsonFormat;
import com.fasterxml.jackson.annotation.JsonInclude;
import com.fasterxml.jackson.annotation.JsonProperty;

import java.time.LocalDate;
```

```
import java.util.List;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class AlumnoDTO {

    @JsonProperty("id")
    private String identificador;
    private String nombre;
    private String apellidos;
    private String dni;

    @JsonFormat(pattern = "yyyy-MM-dd")
    private LocalDate fechaNacimiento;

    private String curso;
    private List<String> documentos;

    public AlumnoDTO() {
    }

    public AlumnoDTO(
        String identificador,
        String nombre,
        String apellidos,
        String dni,
        LocalDate fechaNacimiento,
        String curso) {
        this.identificador = identificador;
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.dni = dni;
        this.fechaNacimiento = fechaNacimiento;
        this.curso = curso;
    }

    public String getIdentificador() {
        return identificador;
    }

    public void setIdentificador(String identificador) {
        this.identificador = identificador;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellidos() {
        return apellidos;
    }

    public void setApellidos(String apellidos) {
        this.apellidos = apellidos;
    }
}
```

```
}

public String getDni() {
    return dni;
}

public void setDni(String dni) {
    this.dni = dni;
}

public LocalDate getFechaNacimiento() {
    return fechaNacimiento;
}

public void setFechaNacimiento(LocalDate fechaNacimiento) {
    this.fechaNacimiento = fechaNacimiento;
}

public String getCurso() {
    return curso;
}

public void setCurso(String curso) {
    this.curso = curso;
}

public List<String> getDocumentos() {
    return documentos;
}

public void setDocumentos(List<String> documentos) {
    this.documentos = documentos;
}
}
```

Por qué reutilizamos las anotaciones de 1.3

@JsonProperty("id") desacopla el nombre Java del contrato JSON. @JsonFormat fija la fecha ISO. @JsonInclude(NON_NULL) evita enviar documentos cuando no tiene valor. No estamos aprendiendo anotaciones nuevas aquí: estamos reutilizando conscientemente el contrato JSON de 1.3 para diseñar otro recurso.

Comprueba compilación:

```
./mvnw test
```

Pregunta

¿Qué ventaja aporta que AlumnoDT0 y ExpedienteDT0 sigan las mismas convenciones JSON?

Respuesta razonada

El cliente puede aprender una regla y aplicarla a toda la API. Si un identificador se expone como id y las fechas usan yyyy-MM-dd en un recurso, mantener la misma convención en los demás reduce errores, documentación y lógica especial en consumidores.

Paso 3 - Crear AlumnoController con GET colección y POST básico

Crea:

M1/proyecto/src/main/java/es/mecd/demo/miproyecto/controller/AlumnoController.java

Empieza con este estado:

```
package es.mecd.demo.miproyecto.controller;

import es.mecd.demo.miproyecto.dto.AlumnoDTO;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.time.LocalDate;
import java.util.List;

@RestController
@RequestMapping("/api/v1/alumnos")
public class AlumnoController {

    private final List<AlumnoDTO> alumnos = List.of(
        new AlumnoDTO(
            "1", "Ana", "García López", "DNI-DEMO-01",
            LocalDate.of(2010, 5, 12), "5º Primaria"),
        new AlumnoDTO(
            "2", "Luis", "Pérez Ruiz", "DNI-DEMO-02",
            LocalDate.of(2009, 9, 3), "6º Primaria")
    );

    @GetMapping
    public List<AlumnoDTO> listar() {
        return alumnos;
    }

    @PostMapping
    public AlumnoDTO crear(@RequestBody AlumnoDTO dto) {
        return dto;
    }
}
```

Los valores DNI-DEMO-01 y DNI-DEMO-02 son datos explícitamente ficticios para el curso; en este punto no existe todavía validación de DNI.

Qué significa este estado

@RequestMapping("/api/v1/alumnos") establece el recurso y la versión. @GetMapping sin ruta adicional corresponde a la colección. @PostMapping usa la misma URL con otra semántica HTTP.

El POST **sólo devuelve el DTO recibido**. No lo guarda. La lista está creada con List.of(...) y es inmutable. Esa limitación es deliberada en 1.4: primero estudiaremos el contrato REST; 1.5 añadirá persistencia en memoria.

Pregunta

¿Por qué no deberíamos añadir ya alumnos.add(dto) a este paso?

Respuesta razonada

Porque estaríamos adelantando una decisión que la secuencia pedagógica reserva para el CRUD de 1.5. En 1.4 queremos observar el contrato HTTP de creación independientemente de la persistencia. Además, `List.of(...)` es inmutable, por lo que añadir directamente produciría `UnsupportedOperationException`.

Paso 4 - Arrancar y probar el GET de colección

Arranca:

```
./mvnw spring-boot:run
```

En otra terminal:

```
curl -i http://localhost:8080/api/v1/alumnos
```

Debes obtener 200 OK, Content-Type: application/json y un array con dos alumnos.

Comprueba varias propiedades:

- el identificador externo se llama id;
- fechaNacimiento tiene formato yyyy-MM-dd;
- documentos no aparece cuando es null;
- el resultado es una colección JSON, no un texto construido a mano.

Pregunta

¿Por qué una colección se representa como array aunque sólo tuviera un alumno?

Respuesta razonada

Porque el contrato del endpoint es “colección de alumnos”. La cardinalidad actual no debe cambiar el tipo estructural de la respuesta. Un cliente puede tratar siempre la respuesta como lista, tenga cero, uno o muchos elementos.

Paso 5 - Probar el POST de creación y observar el 200 inicial

Sin modificar todavía el controlador, ejecuta:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{"nombre":"María","apellidos":"López","dni":"DNI-DEMO-03","fechaNacimiento":"2011-03-20","cur
```

El cuerpo debe volver serializado, pero observa el código: con un DTO devuelto directamente Spring responde normalmente 200 OK.

Vuelve a ejecutar:

```
curl http://localhost:8080/api/v1/alumnos
```

El nuevo alumno **no aparece**. El POST aún no persiste.

Pregunta

¿Que el POST devuelva el JSON enviado significa que el recurso se ha guardado?

Respuesta razonada

No. Una respuesta puede contener el mismo objeto sin que haya ninguna persistencia. Hay que distinguir serialización, código de estado y efecto sobre el estado del servidor. En este paso sólo estamos haciendo eco del DTO.

Paso 6 - Analizar por qué 200 no describe una creación

200 OK indica que la petición se ha procesado correctamente, pero HTTP dispone de un código más específico para una creación: 201 Created.

Compara conceptualmente:

```
POST /api/v1/alumnos -> 200 OK      # éxito genérico
POST /api/v1/alumnos -> 201 Created # creación declarada explícitamente
```

El cuerpo puede ser idéntico; lo que cambia es el contrato HTTP.

No confundas esta mejora con persistencia. En el siguiente paso cambiaremos el **status**, no la lista.

Pregunta

¿Por qué merece la pena distinguir 200 y 201 si ambos están en la familia 2xx?

Respuesta razonada

Porque un cliente no debería deducir la semántica leyendo texto o conociendo implementación interna. 201 comunica explícitamente que la operación representa una creación. Los códigos de estado son parte de la interfaz pública.

Paso 7 - Corregir el POST para devolver 201 Created

Añade los imports:

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
```

Y sustituye el método crear por:

```
@PostMapping
public ResponseEntity<AlumnoDTO> crear(@RequestBody AlumnoDTO dto) {
    return ResponseEntity.status(HttpStatus.CREATED).body(dto);
}
```

Reinicia y repite el POST:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"María","apellidos":"López","dni":"DNI-DEMO-03","fechaNacimiento":"2011-03-20","cur"
```

Ahora debes ver 201.

Vuelve a listar. El alumno continúa sin guardarse: el objetivo de este paso era corregir el contrato HTTP sin introducir todavía persistencia.

Pregunta

¿Qué aporta ResponseEntity<AlumnoDTO> frente a devolver directamente AlumnoDTO?

Respuesta razonada

Permite controlar la respuesta HTTP completa, especialmente el status y, cuando lo necesitemos, cabeceras. El cuerpo sigue siendo un AlumnoDTO, pero ya no dependemos sólo del status por defecto de Spring MVC.

Paso 8 - Añadir el endpoint GET individual

Añade:

```
import org.springframework.web.bind.annotation.PathVariable;

import java.util.Optional;
```

Y el método:

```
@GetMapping("/{id}")
public ResponseEntity<AlumnoDTO> consultar(@PathVariable String id) {
    Optional<AlumnoDTO> encontrado = alumnos.stream()
        .filter(a -> a.getIdentificador().equals(id))
        .findFirst();

    return encontrado
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
```

Prueba un ID existente:

```
curl -i http://localhost:8080/api/v1/alumnos/1
```

Debe devolver 200 y el alumno.

Qué hace @PathVariable

En la URL /api/v1/alumnos/1, el fragmento 1 forma parte del path. @PathVariable String id lo entrega al método como parámetro Java. El método busca el alumno y utiliza ResponseEntity porque existen dos resultados HTTP válidos: 200 o 404.

Pregunta

¿Por qué consultar necesita controlar el status mientras listar puede devolver directamente una lista?

Respuesta razonada

Una colección válida existe aunque esté vacía y puede responder 200 con []. Un recurso individual concreto puede no existir; esa ausencia debe expresarse como 404. ResponseEntity permite representar ambos casos de forma explícita.

Paso 9 - Probar un ID inexistente

Ejecuta:

```
curl -i http://localhost:8080/api/v1/alumnos/999
```

Debes observar 404 Not Found.

No fijas en tus tests todos los campos del cuerpo de error por defecto de Spring Boot como si fueran nuestro contrato propio. Lo estable que estamos enseñando aquí es el status 404 del recurso inexistente.

Compara:

```
curl -i http://localhost:8080/api/v1/alumnos/1
curl -i http://localhost:8080/api/v1/alumnos/999
```

Misma familia de URL, distinto resultado según la existencia del recurso.

Pregunta

¿Por qué no devolvemos 200 con cuerpo null para el alumno 999?

Respuesta razonada

Porque la URL identifica un recurso individual que no existe. Un 200 afirma éxito de la consulta del recurso; 404 comunica explícitamente la ausencia y permite al cliente tomar decisiones sin interpretar un cuerpo ambiguo.

Paso 10 - Auditar el diseño actual de la API

Revisa lo construido:

Aspecto	Estado	Evidencia
URL en plural	correcto	/api/v1/alumnos
minúsculas	correcto	alumnos
versionado	correcto	/api/v1/
GET colección	correcto	GET /api/v1/alumnos
POST creación	correcto en contrato	POST /api/v1/alumnos -> 201
GET individual	correcto	GET /api/v1/alumnos/{id} -> 200/404
ID en path	correcto	/alumnos/1
DTO con Jackson	correcto	id, fecha ISO, no nulos omitidos
persistencia de POST	pendiente	se completa en 1.5
PUT/PATCH/DELETE	pendiente	se completa en 1.5

La API es deliberadamente incompleta como CRUD, pero las operaciones existentes ya deben respetar sus convenciones.

Pregunta

¿Una API tiene que implementar todos los métodos HTTP para poder estar bien diseñada?

Respuesta razonada

No. Debe implementar las operaciones que necesita el dominio y hacerlo con semántica coherente. Una API parcial puede estar bien diseñada; lo incorrecto sería fingir soporte para operaciones que no existen o usar rutas y códigos inconsistentes.

Paso 11 - Reconocer errores comunes de diseño

Usa esta tabla como auditoría rápida:

Problema	Diseño problemático	Alternativa coherente
verbo en URL	/api/getAlumnos	GET /api/v1/alumnos
singular para colección	/api/v1/alumno	/api/v1/alumnos
mayúsculas	/api/v1/Alumnos	/api/v1/alumnos
guion bajo	/api/v1/tipos_alumno	/api/v1/tipos-alumno
ID como filtro	/api/v1/alumnos?id=1	/api/v1/alumnos/1
sin versión	/api/alumnos	/api/v1/alumnos
POST con éxito genérico	ResponseEntity.ok(dto)	status(CREATED).body(dto)
no encontrado como éxito	ResponseEntity.ok(null)	notFound().build()
extensión en ruta	/api/v1/alumnos.json	/api/v1/alumnos

Diagnóstico operativo

Si recibes 404, confirma ruta e ID. Si recibes 405, revisa el método HTTP. Si recibes 415, revisa Content-Type. Si recibes 400 al enviar JSON, valida sintaxis, tipos y formato de fecha antes de cambiar el controlador.

Pregunta

¿Qué error de diseño de la tabla puede seguir “funcionando” técnicamente y, aun así, merece corregirse?

Respuesta razonada

Prácticamente todos. Por ejemplo /api/getAlumnos puede mapearse y responder 200, pero mezcla la acción con la identidad del recurso. La calidad del diseño de una API no se reduce a “el servidor responde”.

Paso 12 - Reto resuelto: filtrar por curso con query parameter

El recurso sigue siendo la colección /api/v1/alumnos, pero queremos permitir una vista filtrada. Añade:

```
import org.springframework.web.bind.annotation.RequestParam;
```

Y sustituye listar() por:

```
@GetMapping
public List<AlumnoDTO> listar(@RequestParam(required = false) String curso) {
    if (curso == null || curso.isBlank()) {
        return alumnos;
    }

    return alumnos.stream()
        .filter(a -> a.getCurso().equalsIgnoreCase(curso))
        .toList();
}
```

Prueba sin filtro:

```
curl http://localhost:8080/api/v1/alumnos
```

Deben aparecer los dos alumnos iniciales.

Para valores con espacios o el carácter °, deja que curl codifique el parámetro:

```
curl -G http://localhost:8080/api/v1/alumnos \
  --data-urlencode "curso=5º Primaria"
```

Debe aparecer sólo Ana.

Prueba una categoría sin coincidencias:

```
curl -G http://localhost:8080/api/v1/alumnos \
  --data-urlencode "curso=3º Primaria"
```

Debe devolver:

```
[]
```

con 200.

Pregunta

¿Por qué curso va en query parameter y no como /api/v1/alumnos/curso/5º Primaria?

Respuesta razonada

Porque curso es un criterio opcional aplicado a la colección, no la identidad de un alumno. La query expresa naturalmente “la colección de alumnos filtrada por curso” y permite combinar más criterios en el futuro sin crear una jerarquía artificial de rutas.

Resultado esperado al cerrar 1.4

El snapshot debe conservar íntegro 1.3 y añadir exactamente:

```
src/main/java/es/mecd/demo/miproyecto/dto/AlumnoDTO.java
src/main/java/es/mecd/demo/miproyecto/controller/AlumnoController.java
```

AlumnoController debe exponer:

```
GET /api/v1/alumnos          -> 200, lista; filtro opcional curso
GET /api/v1/alumnos/{id}     -> 200 o 404
POST /api/v1/alumnos         -> 201, devuelve el DTO recibido
```

El POST todavía **no añade** el DTO a la lista. La lista continúa siendo inmutable mediante `List.of(...)`. Esa limitación se resolverá deliberadamente en 1.5.

Verificación final:

```
./mvnw test
./mvnw -DskipTests package
curl http://localhost:8080/api/v1/alumnos
curl -i http://localhost:8080/api/v1/alumnos/1
curl -i http://localhost:8080/api/v1/alumnos/999
```

Y confirma que los endpoints anteriores siguen funcionando:

```
curl http://localhost:8080/hola
curl http://localhost:8080/adios
curl http://localhost:8080/api/v1/expedientes/ejemplo
```

Práctica 1.5 - Primer CRUD con DTOs

Objetivo práctico

Completar el CRUD de Alumno iniciado en 1.4. El controlador dejará de ser sólo una demostración de contrato REST: el POST guardará en memoria, PUT reemplazará un alumno, PATCH modificará campos concretos y DELETE eliminará. Cerraremos el módulo combinando filtrado y ordenación opcionales.

El almacenamiento sigue siendo **en memoria**. Al reiniciar la aplicación se recuperan únicamente los dos alumnos iniciales. Esta limitación es deliberada: persistencia real, servicios y repositorios llegan en módulos posteriores.

Estado de partida

Conserva íntegros los puntos 1.1-1.4. En M1/proyecto deben seguir funcionando:

```
GET /hola
GET /adios
GET /api/v1/expedientes/ejemplo
POST /api/v1/expedientes/eco
GET /api/v1/alumnos
GET /api/v1/alumnos/{id}
POST /api/v1/alumnos
```

En 1.4 el POST de alumnos devuelve 201, pero **no persiste**. Ese será el primer cambio de 1.5.

Desde la carpeta M1/proyecto, en Linux/macOS:

```
./mvnw spring-boot:run
```

En Windows:

```
mvnw.cmd spring-boot:run
```

Desde IntelliJ IDEA

Ejecuta MiProyectoApplication con Run. Tras modificar el controlador, reinicia si la recarga automática no aplica el cambio.

Desde Eclipse

Ejecuta MiProyectoApplication con Run As > Java Application y usa una terminal para las pruebas HTTP.

Desde VS Code

Ejecuta la clase principal desde el soporte Java/Spring o usa el terminal integrado con Maven Wrapper.

Paso 1 - Repasar el estado actual

Antes de modificar nada, demuestra qué existe ya:

```
curl -i http://localhost:8080/api/v1/alumnos
curl -i http://localhost:8080/api/v1/alumnos/1
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{"nombre":"María","apellidos":"López","dni":"DNI-DEMO-03","fechaNacimiento":"2011-03-20","cur
```

El GET de colección debe mostrar dos alumnos. El GET individual debe devolver 200 para /1. El POST debe devolver 201 y el DTO recibido.

Ahora vuelve a listar:

```
curl http://localhost:8080/api/v1/alumnos
```

Siguen existiendo sólo dos alumnos. Ésta es la limitación que vamos a resolver.

Pregunta

¿Qué falta para que el POST sea una creación funcional y no sólo una respuesta 201 con el cuerpo recibido?

Respuesta razonada

Hace falta modificar un almacenamiento mutable. El controlador debe asignar identidad al nuevo recurso, añadirlo a la colección y permitir recuperarlo con GET posteriormente. El status por sí solo no persiste nada.

Paso 2 - Convertir la lista en mutable

Abre:

```
M1/proyecto/src/main/java/es/mecd/demo/miproyecto/controller/AlumnoController.java
```

Añade:

```
import java.util.ArrayList;
```

Y cambia la inicialización:

```
private final List<AlumnoDTO> alumnos = new ArrayList<> (List.of(
    new AlumnoDTO(
        "1", "Ana", "García López", "DNI-DEMO-01",
        LocalDate.of(2010, 5, 12), "5º Primaria"),
    new AlumnoDTO(
        "2", "Luis", "Pérez Ruiz", "DNI-DEMO-02",
        LocalDate.of(2009, 9, 3), "6º Primaria")
));
```

List.of(...) crea una lista inmutable. Envolverla en new ArrayList<>(...) crea una colección mutable con los mismos datos iniciales.

Comprueba que nada anterior se ha roto:

```
./mvnw test
```

Pregunta

¿Por qué no basta con mantener List.of(...) y llamar después a add?

Respuesta razonada

Porque las listas creadas con List.of no admiten cambios estructurales. add, remove o operaciones equivalentes lanzarían UnsupportedOperationException. El CRUD necesita una colección mutable mientras usemos este almacenamiento pedagógico.

Paso 3 - Modificar el POST para que guarde el alumno

Sustituye crear por:

```
@PostMapping
public ResponseEntity<AlumnoDTO> crear(@RequestBody AlumnoDTO dto) {
    int siguienteId = alumnos.stream()
        .map(AlumnoDTO::getIdificador)
        .filter(id -> id != null && id.matches("\\d+"))
        .mapToInt(Integer::parseInt)
        .max()
        .orElse(0) + 1;

    dto.setIdentificador(String.valueOf(siguienteId));
    alumnos.add(dto);

    return ResponseEntity.status(HttpStatus.CREATED).body(dto);
}
```

La fuente utiliza una generación de ID deliberadamente simple para obtener 3 tras los dos alumnos iniciales. Aquí usamos máximo + 1 en vez de size()+1: sigue siendo una simplificación en memoria, pero evita una colisión inmediata si antes se ha borrado un elemento.

No es un generador de IDs de producción. Dos peticiones concurrentes podrían calcular el mismo valor; una base de datos o un repositorio real resolverán esa responsabilidad más adelante.

Reinicia y crea:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"Pedro","apellidos":"Sánchez","dni":"DNI-DEMO-03","fechaNacimiento":"2010-07-15", "c
```

Debe devolver 201 Created y:

```
{"id":"3", ...}
```

Comprueba persistencia en memoria:

```
curl http://localhost:8080/api/v1/alumnos/3
curl http://localhost:8080/api/v1/alumnos
```

El alumno 3 debe existir hasta que reinicies o lo borres.

Pregunta

¿Por qué ignoramos un id que pudiera enviar el cliente en el POST y asignamos uno en el servidor?

Respuesta razonada

Porque en este diseño la identidad del nuevo recurso la controla el servidor. Permitir que cualquier cliente elija libremente el ID facilita colisiones e inconsistencias. La estrategia concreta de generación es simplificada, pero la responsabilidad está situada en el lado correcto.

Paso 4 - Añadir el endpoint PUT

Añade el import:

```
import org.springframework.web.bind.annotation.PutMapping;
```

Y el método:

```
@PutMapping("/{id}")
public ResponseEntity<AlumnoDTO> actualizar(
    @PathVariable String id,
    @RequestBody AlumnoDTO dto) {

    for (int i = 0; i < alumnos.size(); i++) {
        if (alumnos.get(i).getIdificador().equals(id)) {
            dto.setIdentificador(id);
            alumnos.set(i, dto);
            return ResponseEntity.ok(dto);
        }
    }

    return ResponseEntity.notFound().build();
}
```

Prueba un alumno existente:

```
curl -i -X PUT http://localhost:8080/api/v1/alumnos/1 \
-H "Content-Type: application/json" \
-d '{"nombre":"Ana","apellidos":"García López","dni":"DNI-DEMO-01","fechaNacimiento":"2010-05-12"}
```

Debe devolver 200. Comprueba que el ID sigue siendo 1 y que el recurso completo ha sido reemplazado.

Prueba un ID inexistente:

```
curl -i -X PUT http://localhost:8080/api/v1/alumnos/999 \
-H "Content-Type: application/json" \
-d '{"nombre":"Nadie","apellidos":"Ejemplo","dni":"DNI-DEMO-X","fechaNacimiento":"2010-01-01","cu"
```

Debe devolver 404.

Pregunta

¿Por qué `dto.setIdentificador(id)` usa el ID de la URL aunque el JSON pudiera contener otro?

Respuesta razonada

Porque la URL identifica el recurso que estamos reemplazando. Si permitiéramos que el cuerpo cambiara silenciosamente su identidad, una petición a /alumnos/1 podría terminar modificando la identidad a otro valor. El path es la autoridad para la operación.

Paso 5 - Añadir el endpoint PATCH

Añade:

```
import org.springframework.web.bind.annotation.PatchMapping;
```

```
import java.util.Map;
```

Y el método:

```
@PatchMapping("/{id}")
public ResponseEntity<AlumnoDTO> actualizarParcial(
    @PathVariable String id,
    @RequestBody Map<String, Object> cambios) {

    for (AlumnoDTO alumno : alumnos) {
        if (alumno.getIdentificador().equals(id)) {
            if (cambios.containsKey("nombre")) {
                alumno.setNombre((String) cambios.get("nombre"));
            }
            if (cambios.containsKey("apellidos")) {
                alumno.setApellidos((String) cambios.get("apellidos"));
            }
            if (cambios.containsKey("dni")) {
                alumno.setDni((String) cambios.get("dni"));
            }
            if (cambios.containsKey("curso")) {
                alumno.setCurso((String) cambios.get("curso"));
            }
            return ResponseEntity.ok(alumno);
        }
    }

    return ResponseEntity.notFound().build();
}
```

Este Map<String, Object> es una simplificación pedagógica. No da el mismo tipado ni validación que un DTO específico de PATCH. Lo usamos sólo para ver claramente qué significa actualizar campos presentes y conservar los ausentes.

Prueba:

```
curl -i -X PATCH http://localhost:8080/api/v1/alumnos/1 \
-H "Content-Type: application/json" \
-d '{"curso":"6º Primaria B"}'
```

Sólo debe cambiar curso.

Prueba otro campo:

```
curl -i -X PATCH http://localhost:8080/api/v1/alumnos/1 \
-H "Content-Type: application/json" \
-d '{"nombre":"Ana María"}'
```

Pregunta

¿Qué tendrías que enviar con PUT para cambiar sólo el curso sin perder el resto del estado?

Respuesta razonada

Tendrías que enviar la representación completa del recurso con todos sus campos y el nuevo curso. PATCH permite expresar directamente que sólo cambia un subconjunto. Nuestro Map hace visible esa diferencia, aunque más adelante usaremos diseños más tipados.

Paso 6 - Añadir el endpoint DELETE

Añade:

```
import org.springframework.web.bind.annotation.DeleteMapping;
```

Y:

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable String id) {
    boolean eliminado = alumnos.removeIf(
        a -> a.getIdentificador().equals(id));

    if (eliminado) {
        return ResponseEntity.noContent().build();
    }

    return ResponseEntity.notFound().build();
}
```

Prueba:

```
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/2
```

Debe devolver 204 No Content.

Repite exactamente la misma petición:

```
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/2
```

Ahora devuelve 404 porque el recurso ya no existe.

Pregunta

¿Sigue siendo DELETE idempotente si la primera respuesta es 204 y la segunda 404?

Respuesta razonada

Sí. Tras una o varias ejecuciones el estado final es el mismo: el alumno 2 no existe. Idempotencia se refiere al efecto sobre el estado, no a que todas las respuestas deban tener el mismo código.

Paso 7 - Probar el CRUD completo con curl

Reinicia primero la aplicación para recuperar el estado inicial de dos alumnos. Ejecuta esta secuencia en orden.

1. Listar

```
curl http://localhost:8080/api/v1/alumnos
```

2. Consultar

```
curl http://localhost:8080/api/v1/alumnos/1
```

3. Crear

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
-H "Content-Type: application/json" \
-d '{"nombre":"Pedro","apellidos":"Sánchez","dni":"DNI-DEM0-03","fechaNacimiento":"2010-07-15","c
```

Comprueba 201 e id: "3".

4. Consultar el creado

```
curl http://localhost:8080/api/v1/alumnos/3
```

5. Reemplazar con PUT

```
curl -i -X PUT http://localhost:8080/api/v1/alumnos/3 \
-H "Content-Type: application/json" \
-d '{"nombre":"Pedro","apellidos":"Sánchez","dni":"DNI-DEM0-03","fechaNacimiento":"2010-07-15","c
```

6. Modificar parcialmente con PATCH

```
curl -i -X PATCH http://localhost:8080/api/v1/alumnos/3 \
-H "Content-Type: application/json" \
-d '{"curso":"6º Primaria B"}'
```

7. Eliminar

```
curl -i -X DELETE http://localhost:8080/api/v1/alumnos/3
```

Debe devolver 204.

8. Verificar ausencia

```
curl -i http://localhost:8080/api/v1/alumnos/3
```

Debe devolver 404.

Pregunta

¿Qué prueba del ciclo demuestra mejor que el POST ha pasado de “eco” a creación real?

Respuesta razonada

Poder consultar después /api/v1/alumnos/3. El 201 es parte del contrato, pero el GET posterior demuestra que el estado del servidor cambió y que el recurso puede recuperarse por su identidad.

Paso 8 - Observar los códigos de estado

Recopila las evidencias del CRUD:

Operación	Código esperado
GET colección	200
GET individual existente	200
GET individual inexistente	404
POST crear	201

Operación	Código esperado
PUT existente	200
PUT inexistente	404
PATCH existente	200
PATCH inexistente	404
DELETE existente	204
DELETE inexistente	404

Añade también los errores de protocolo que ya conocemos:

Situación	Código esperado
JSON mal formado	400
método no soportado	405
cuerpo con media type no soportado	415

Pregunta

¿Por qué conviene verificar los códigos de estado por separado del cuerpo JSON?

Respuesta razonada

Porque expresan dimensiones distintas del contrato. El status resume el resultado de la operación para cualquier cliente HTTP; el cuerpo representa datos o detalles adicionales. Un cuerpo correcto con un status incorrecto sigue siendo una API incorrecta.

Paso 9 - Probar un error de formato

Provoca un JSON inválido:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Pedro","apellidos":"Sánchez",}'
```

Debe devolver 400 Bad Request.

No acoples tu comprobación al texto exacto del parser o a todos los campos del JSON de error de Spring Boot. El contrato estable que queremos demostrar aquí es el status 400.

También puedes comprobar una fecha incompatible:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Pedro","fechaNacimiento":"15/07/2010"}'
```

Debe fallar al deserializar LocalDate según nuestro contrato yyyy-MM-dd.

Pregunta

¿Por qué este 400 puede aparecer sin que escribamos un if específico en crear()?

Respuesta razonada

Porque Spring MVC y Jackson deben convertir primero el cuerpo HTTP en AlumnoDTO. Si el JSON no se puede parsear o convertir a los tipos declarados, la petición falla antes de que el método reciba un DTO válido.

Paso 10 - Probar un método no soportado

La URL individual admite GET, PUT, PATCH y DELETE, pero no POST. Ejecuta:

```
curl -i -X POST http://localhost:8080/api/v1/alumnos/1 \
  -H "Content-Type: application/json" \
  -d '{}'
```

Debe devolver 405 Method Not Allowed.

Compara con:

```
curl -i http://localhost:8080/api/v1/alumnos/999
```

Éste devuelve 404: el método GET sí existe para esa forma de URL, pero el alumno 999 no.

Pregunta

¿Qué información diferente aporta 405 respecto a 404?

Respuesta razonada

405 indica que la ruta se reconoce para otros métodos pero el método solicitado no está soportado. 404, en nuestro GET individual, expresa que el recurso identificado no existe. Son fallos distintos y el cliente puede reaccionar de manera distinta.

Paso 11 - Errores comunes del ejercicio

Usa esta tabla antes de cambiar código al azar:

Síntoma	Causa probable	Comprobación / solución
UnsupportedOperationException al crear/borrar	sigue List.of(...)	usar new ArrayList<>(List.of(...))
POST devuelve 201 pero el GET no muestra el nuevo	falta alumnos.add(dto)	comprobar el método crear
IDs se repiten	generación demasiado simple	revisar estrategia; recordar que sigue siendo didáctica
PUT cambia el ID	se confía en el cuerpo	imponer dto.setIdentificador(id)
PUT/PATCH de ID inexistente	búsqueda no contempla ausencia	devolver 404
PATCH no cambia un campo	clave no contemplada en el Map	revisar nombre y conversión
DELETE no elimina	ID no coincide	inspeccionar removeIf
POST a /{id} devuelve 405	método equivocado	usar PUT/PATCH según intención
POST/PUT/PATCH devuelve 415	Content-Type ausente/incorrecto	enviar application/json
POST/PUT devuelve 400	JSON o tipos incompatibles	validar sintaxis y fechas

Límite importante

Un Map<String, Object> permite enseñar PATCH con poco código, pero no es un contrato de entrada robusto. Tampoco ArrayList dentro del controlador es una arquitectura final. Esas limitaciones son parte del aprendizaje: en M2 empezaremos a separar responsabilidades.

Pregunta

¿Por qué es útil conocer estas limitaciones antes de introducir service y repository?

Respuesta razonada

Porque podremos distinguir un refactor arquitectónico de un cambio de comportamiento. Sabremos qué contrato debe seguir funcionando mientras movemos responsabilidades a otras capas.

Paso 12 - Reto resuelto: filtrar por curso y ordenar

El filtro curso de 1.4 permanece. Añadiremos un segundo query parameter realmente opcional: sort.

Sustituye listar() por:

```
@GetMapping
public List<AlumnoDTO> listar(
    @RequestParam(required = false) String curso,
    @RequestParam(required = false) String sort) {

    var stream = alumnos.stream();

    if (curso != null && !curso.isBlank()) {
        stream = stream.filter(
            a -> a.getCurso().equalsIgnoreCase(curso));
    }

    if ("nombre".equalsIgnoreCase(sort)) {
        stream = stream.sorted(
            (a, b) -> a.getNombre().compareToIgnoreCase(b.getNombre()));
    } else if ("apellidos".equalsIgnoreCase(sort)) {
        stream = stream.sorted(
            (a, b) -> a.getApellidos().compareToIgnoreCase(b.getApellidos()));
    }

    return stream.toList();
}
```

La fuente histórica muestra defaultValue="nombre" y a la vez describe sort como opcional. Aquí hacemos ambas cosas coherentes: si no hay sort, no ordenamos y conservamos el orden actual de la colección.

Prueba sin parámetros:

```
curl http://localhost:8080/api/v1/alumnos
```

Filtra usando codificación segura:

```
curl -G http://localhost:8080/api/v1/alumnos \
  --data-urlencode "curso=5º Primaria"
```

Ordena por nombre:

```
curl -G http://localhost:8080/api/v1/alumnos \
  --data-urlencode "sort=nombre"
```

Ordena por apellidos:

```
curl -G http://localhost:8080/api/v1/alumnos \
  --data-urlencode "sort=apellidos"
```

Combina ambos:

```
curl -G http://localhost:8080/api/v1/alumnos \
  --data-urlencode "curso=5º Primaria" \
  --data-urlencode "sort=nombre"
```

Un valor de sort no reconocido deja el orden actual sin convertirlo en error en este ejercicio introductorio.

Pregunta

¿Qué ventaja tiene mantener curso y sort como query parameters en lugar de crear rutas nuevas para cada combinación?

Respuesta razonada

El recurso sigue siendo la misma colección. Los parámetros sólo modifican la vista solicitada y pueden combinarse sin multiplicar endpoints como /alumnos-ordenados-por-nombre o /alumnos-5-primaria.

Resultado esperado al cerrar 1.5 y M1

El proyecto debe conservar todo lo construido anteriormente y AlumnoController debe ofrecer estas seis combinaciones principales método/ruta:

GET	/api/v1/alumnos	-> 200; filtro y ordenación opcionales
GET	/api/v1/alumnos/{id}	-> 200 / 404
POST	/api/v1/alumnos	-> 201; asigna ID y guarda en memoria
PUT	/api/v1/alumnos/{id}	-> 200 / 404; reemplazo completo
PATCH	/api/v1/alumnos/{id}	-> 200 / 404; actualización parcial
DELETE	/api/v1/alumnos/{id}	-> 204 / 404

Verificación reproducible:

```
./mvnw test
./mvnw -DskipTests package
java -jar target/mi-proyecto-0.0.1-SNAPSHOT.jar
```

Y después recorre el ciclo CRUD del paso 7.

Comprueba también que no has roto el contenido anterior:

```
curl http://localhost:8080/hola
curl http://localhost:8080/adios
curl http://localhost:8080/api/v1/expedientes/ejemplo
```

Al cerrar M1 debes ser capaz de explicar la cadena completa:

Spring Boot -> HTTP -> JSON/Jackson -> diseño REST -> CRUD

El siguiente módulo podrá refactorizar este comportamiento hacia capas de controlador, servicio y repositorio sin tener que redescubrir qué debe hacer la API.